DEEP LEARNING

PART-I
(ANN IMPLEMENTATION)

↳ **Deep learning**: It is a subset of Machine learning that uses Artificial Neural Network (ANN) with Multiple layers to learn complex patterns directly from data.

Inspired by human brain, where millions of neurons work together to process information.

↳ **Machine Learning (ML) Vs Deep Learning (DL)**

Machine Learning (ML)	Deep Learning (DL)
(1) Manual feature engineering	(1) Automatic feature Learning
(2) Works well on small datasets	(2) Requires large dataset
(3) Faster training	(3) slower training
(4) Less computational power	(4) Need GPUs/TPUs
(5) Simpler Models	(5) Deep Neural Network

Why Deep learning is popular ?

(1.) Huge Amount of Data

Images, videos, text & audio are now available in massive quantities for use.

(2.) Powerful Hardware

GPUs & TPUs can train deep models much faster.

(3.) Automatic Feature Extraction

No need to manually select features

(4.) High Accuracy

Performs well in :

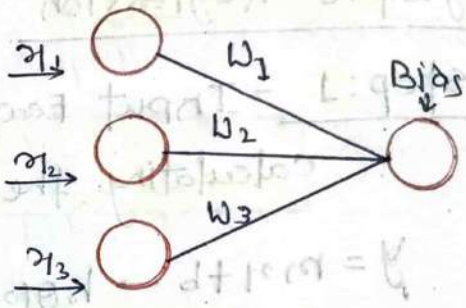
- computer Vision
- NLP (Natural language Processing)
- speech Recognition
- Medical Imaging
- self Driving cars
- Robotics

→ Perceptron

It is the simplest & first artificial neuron & the basic building block of Artificial Neural Network(ANN), It was introduced by Frank Rosenblatt in 1958.

It mimics the working of a biological neuron by -

- Receiving inputs
- Processing them
- Producing an output



↳ Biological Neuron Vs Artificial Neuron

(1) Dendrites Receives signals	(1) Input receive data
(2) cell body processes signals	(2) Weighted sum processes data
(3) Axon sends output	(3) Activation function gives output.

↳ Limitations of a single perceptron

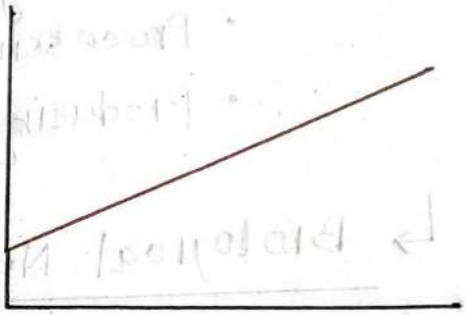
- solves only linearly separable problems.
- cannot solve XOR & other non-linear problems.
- Original perceptron uses only step Activation
- Limited learning capability.

Logistic Regression Vs Perceptron

Age	cholestrol	BP	Heart Disease
28	150	110	1
36	120	90	0
42	180	160	1

Logistic Regression

step:1 - Input Features & calculating the linear equation



$y = mx + b \rightarrow h\theta(x) = \theta_0 + \theta_1 x_1$

$h\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$

step:2 - Apply sigmoid Function

$\sigma(z) = \frac{1}{1+e^{-(z)}}$, output range: 0 to 1

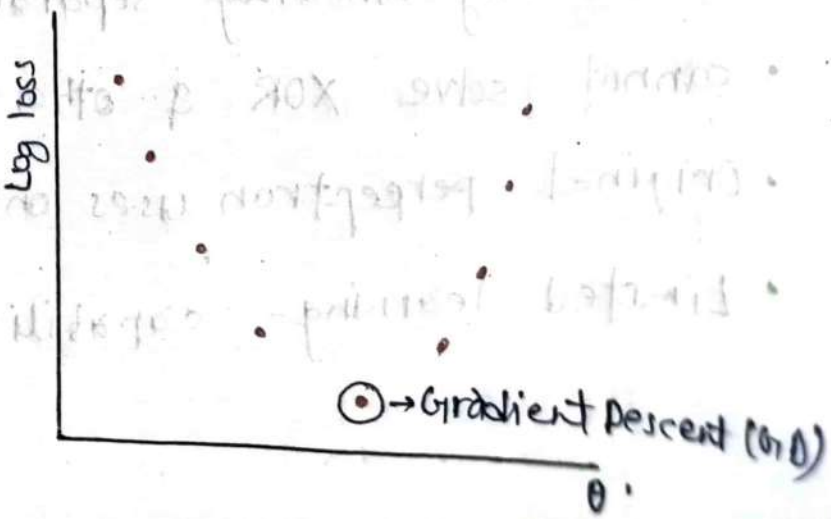
$= z = (\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3)$

step:3 - Log loss function

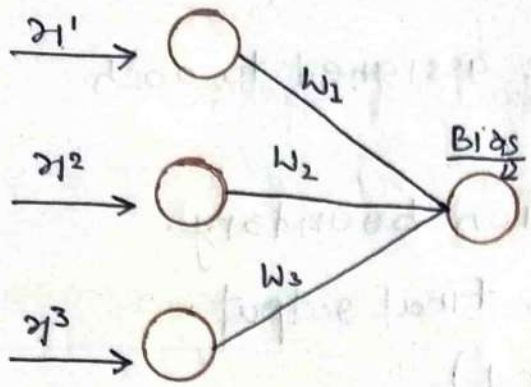
↳ find loss

step:4 - optimizer

$\theta_0 = 0$



Perceptron



Step: 1 - Input features & Intialize weight & Bias

input features - x_1, x_2, x_3

weights - w_1, w_2, w_2

bias = b

Step: 2 - Calculate the weighted Sum(z)

$$z = w_1 x_1 + w_2 x_2 + w_2 x_3 + b$$ → bias

Step: 3:- Apply the Activation function

The original perceptron uses the step Activation function.

if $z \geq 0$ → output = 1

if $z < 0$ → output = 0

Step: 4 - compare with the Actual weight output.

step: 5 - Calculate Error

error = Actual output - Predicted output

Possible Value -

- error = 0 → correct prediction
- error = 1 → Predicted 0 instead of 1
- error = -1 → Predicted 1 instead of 0

Step: 6 - Update weight & Bias

Step: 7 - Repeat steps 3-7 for all training sample over multiple epochs untill - All samples are classified correctly or the maximum no. epochs is reached.

components of perceptron -

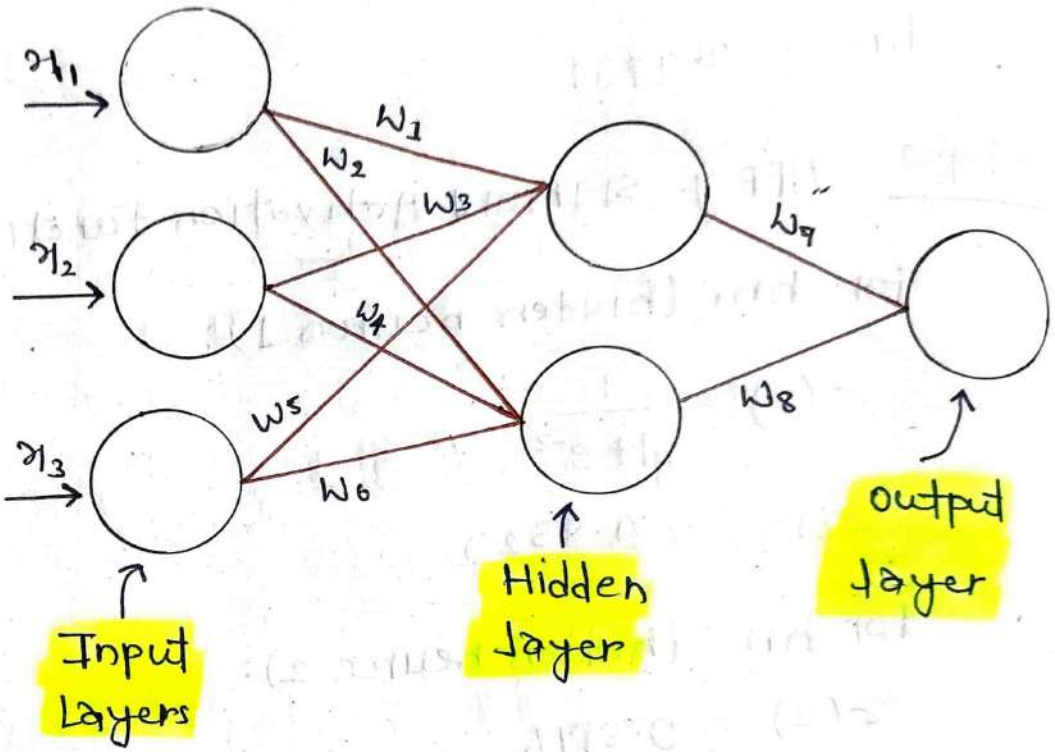
- Inputs (x_1, x_2, x_3) : Features from the dataset
- Weights (w_1, w_2, w_3) - Importance assigned to each input.
- Bias (b) - helps shift the decision boundary.
- Activation Function - Decides the final output.
- Output (y) - Predicted class (0 or 1)

→ ANN (Artificial Neural Network)

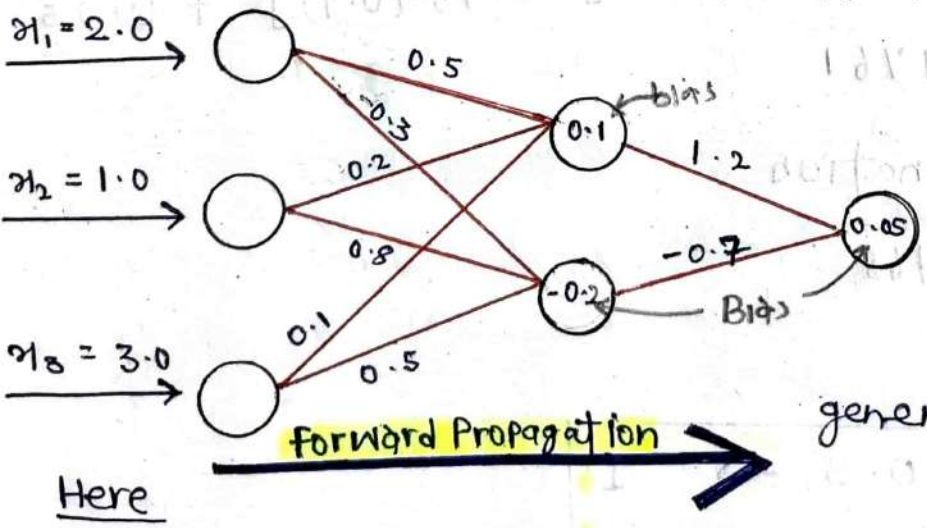
It is an network of inter-connected artificial neurons (perceptrons) arranged in layers. It is designed to learn patterns from data & make predictions.

An ANN consists -

- Input Layer
- Hidden layer(s)
- output layer



→ **Forward propagation** - It is the process in which input data moves forward through the neural network - from input layer, through one or more hidden layer & finally to the output layer - to generate a prediction.



Here

$W_1 = 0.5$, $W_4 = 0.8$, bias $(b_1) = 0.1$
$W_2 = -0.3$, $W_5 = 0.1$, $b_2 = -0.2$
$W_3 = 0.2$, $W_6 = 0.5$, $b_3 = 0.05$
$W_7 = 1.2$, $W_7 = -0.7$

Forward Propagation steps

Step 1: calculate hidden Neuron

$$hn_1 = (x_1(w_1)) + (x_2(w_2)) + (x_3(w_3)) + \text{bias}$$

$$= (2.0)(0.5) + (1.0)(0.2) + (3.0)(0.1) + 0.1$$

$$\approx 1.6$$

$$hn_2 = 0.4761$$

Step 2: Apply sigmoid Activation Function

for hn_1 (hidden neuron 1):

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\sigma(1.2) = 0.8320$$

for hn_2 (hidden neuron 2):

$$\sigma(z) = 0.8176$$

Step 3: calculate output Neuron

$$\text{output Neuron} = [0.8320(1.2)] + [0.8176(0.4)] + 0.05$$

$$= 0.4761$$

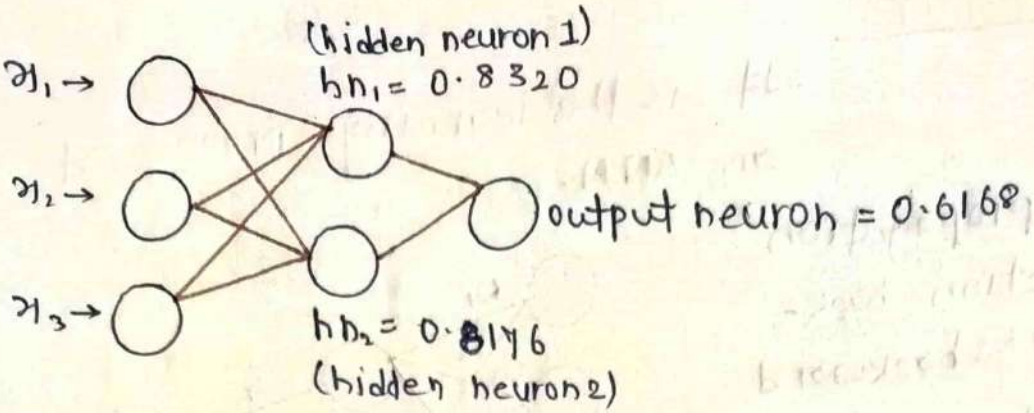
Step 4: Apply sigmoid function

$$\sigma(0.4761) = 0.6168$$

Step 5: Final Prediction

if Probability	≥ 0.5	$\rightarrow$ 1
if probability	< 0.5	$\rightarrow$ 0

Prediction = $0.6168 > 0.5 \Rightarrow y = 1$



↳ Three Important components

(1) Activation Function

Introduces non-linearity into the network & helps neurons learns complex patterns

ex - sigmoid

(2) Loss Function

Measures how far the prediction is from the actual value

ex - Binary cross entropy, Meansquared error.

* Lower loss means better prediction

(3) optimizer

updates the weights & biases to reduce the loss.

ex - gradient descent

* The optimizer works during Backward propagation

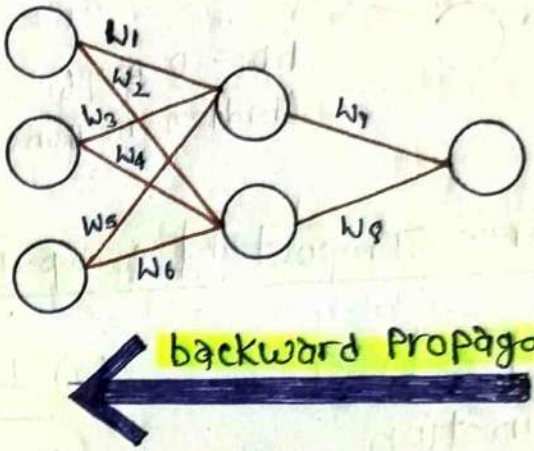
Forward Propagation

Backward Propagation

→ Backward Propagation

It is the learning process of an ANN.

After Forward propagation makes a prediction back-propagation works backward through the network to find the error, calculate how much each weight contribute to that error & update the weights to improve future predictions.



↳ steps of Backward Propagation

Step: 1 - calculate the loss

$$\text{Log loss} = [y \times \log(\hat{y}) + (1-y) \times \log(1-\hat{y})]$$

log loss = 0.494 → from previous example

Actual output (y) = 1

Predicted output ($\hat{y}$) = 0.6168

Step: 2 - calculate Gradients (chain Rule)

It uses chain rule of Derivatives to compute the gradient of the loss with respect to every weight.

ex

$$\frac{\partial L}{\partial w}$$

Step 3: Update the weights

Once the gradient are known, update each weight.

Formula:

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial w}$$

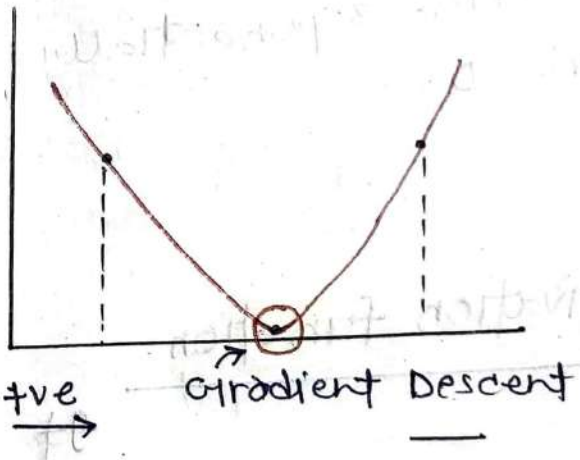
w = weight
η = Learning Rate
$\frac{\partial L}{\partial w}$ = Gradient

The same idea is used to update the bias

Step 4: Optimizer Updates the weights

Optimizer :- It decides how to update the weights efficiently.

ex- Gradient Descent
stochastic Gradient Descent (SGD)
Adam
RMSprop



step: 5 - Repeat the Process

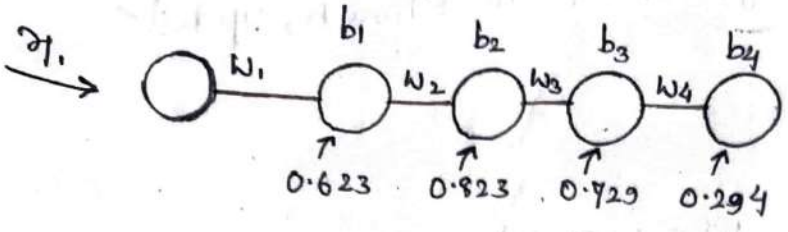
Afterupdating weight -

(i) Run Forward Propagation again
(ii) Calculate New loss
(iii) Run Backward Propagation
(iv) Update the weights again

} Repeat this over many epochs until the loss becomes very small

epochs = Forward + Backword Propagation

→ Vanishing Gradient Problem



It occurs during "Backward Propagation" when gradients becomes extremely small (close to 0) causing the early layers of deep neural network to stop learning.

↳ Cause :

- occurs mainly with sigmoid activation function

$$\sigma = \frac{1}{1+e^{-z}}$$

- During propagation, gradients are Multiplied across many layers using the chain Rule.
- since gradient less than 1, repeated Multiplication makes them shrink exponentially towards 0.

↳ solution

Modern deep learning uses ReLU (Rectified Linear unit) instead of sigmoid in Hidden layers.

RELU Function

$$RELU(x) = \max(0, x)$$

→ Activation function

It is a mathematical function applied to a neuron's weighted sum of inputs that determines its output.

It introduces non-linearly enabling neural Networks to learn complex patterns & relationships in data.

* Non-linearity - It enables a neural network to learn complex patterns that cannot be represented by a straight line.

→ Activation Function Types

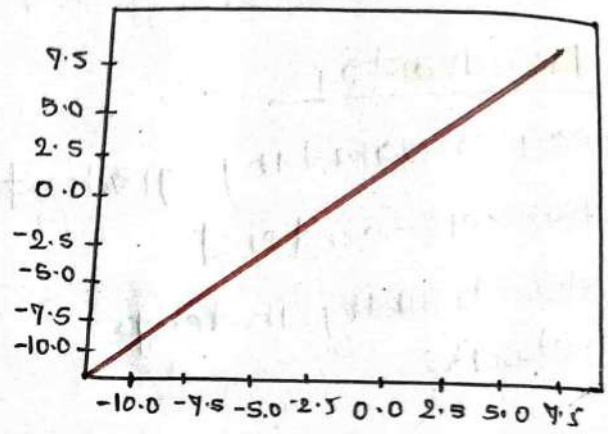
(1.) Linearly Activation Function

It is the simplest activation Function. It returns the input value without applying any transformation or non-linearity.

Formula-

$$f(x) = x$$

- x = Input to the neuron
- $f(x)$ = output of the neuron



↳ Advantages

- simple & fast
- Best for regression Tasks
- Produces unrestricted outputs.

↳ Disadvantages

- No non-linearity
- cannot learn complex patterns
- Not suitable for hidden layers.

↳ Use cases

- House price Prediction
- salary Prediction
- Temperature Prediction
- stock Price Prediction

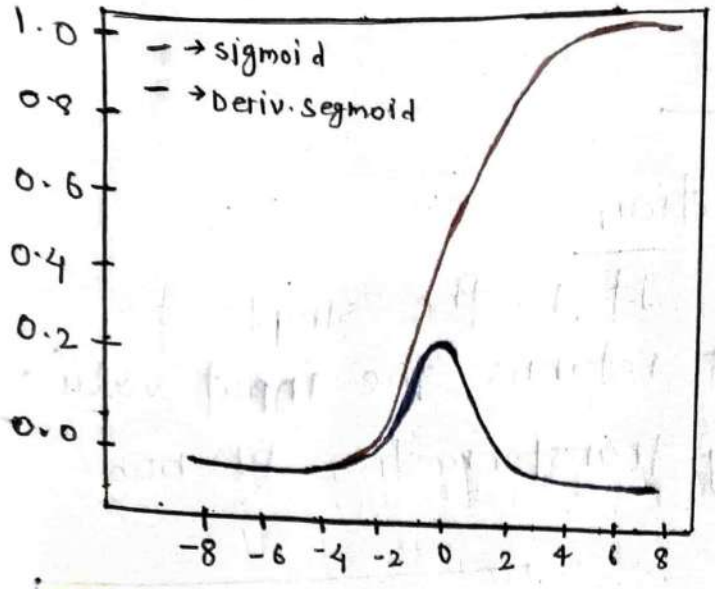
↳ Used in → output layer (Regression)

(2.) Sigmoid Activation Function

It converts any real no. into a value between 0 & 1, making suitable for probability Pridiction.

Formula-

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



↳ **Disadvantages**

- causes vanishing gradient
- Not zero-centered
- slow training in deep networks

↳ **Advantages**

- outputs probabilities (0, 1)
- Ideal for binary classification
- Easy to interpret.

↳ **Used In**

- output layer (Binary classifica

↳ **Use cases**

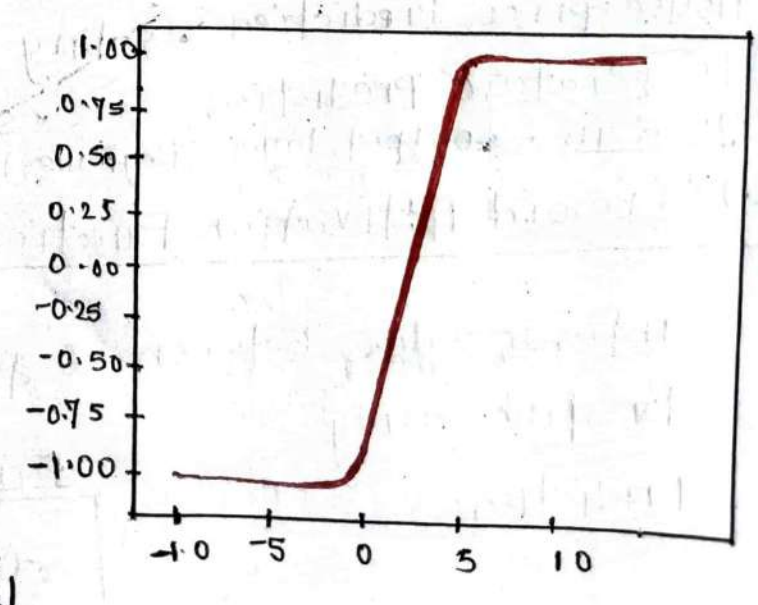
- spam detection
- Disease Prediction
- Fraud detection

(3) Tanh (Hyperbolic Tangent) Activation Function

It is another squashing function like sigmoid, but instead of squeezing values into 0 to 1, it squeezes them into -1 to +1. mean inputs to values between -1 & 1.

Formula

$$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$$



↳ **Advantages**

- zero-centered output.
- Learns faster than sigmoid
- Handles negative values well

↳ Disadvantages

- still suffer from vanishing gradients.
- computationally expensive
- Rarely used in deep hidden layers

↳ Use cases

- older Neural Network
- RNNs (earlier architectures)

↳ Used In

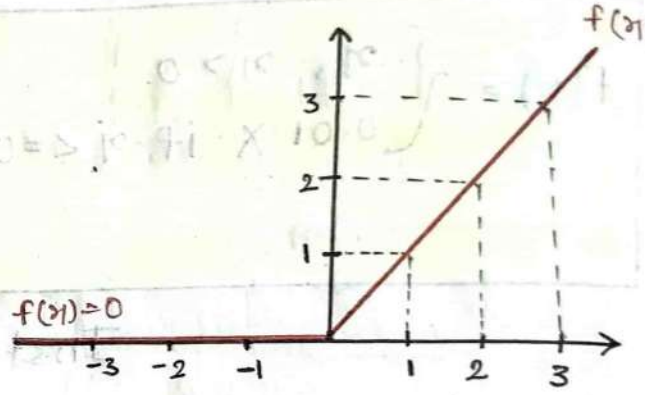
- Hidden Layer (older Networks)

→ (4.) ReLU (Rectified Linear Unit)

It outputs 0 for negative inputs and the input itself for positive inputs.

Formula

$$f(x) = \max(0, x)$$



That means:

- if input $x < 0$ → output = 0
- if input $x > 0$ → output = x

so, it either passes positive values as they are or blocks negative values by turning them into 0.

↳ Advantages

- very fast & simple to compute
- Helps avoid vanishing gradient problem (better than sigmoid & tanh)
- Make training deep Network much faster.

↳ Disadvantages

- Dying ReLU problem

sometimes neurons get stuck at 0 forever if weights update badly

- Not smooth at 0

Not differentiable there, but still works fine in practice.

↳ Use cases

- CNNs
- Deep Neural Networks
- Computer vision

↳ Used In

- Hidden layers

(5) Leaky ReLU

It is a modified version of ReLU that allows a small negative output instead of zero

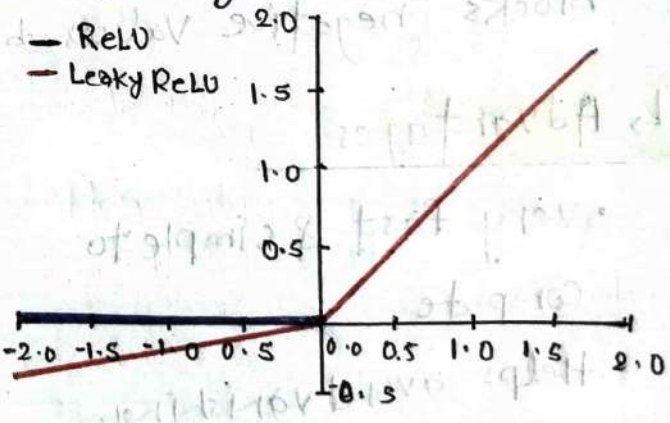
Formula

$$f(x) = \begin{cases} x, & x > 0 \\ 0.01x, & x <= 0 \end{cases} \qquad f(x) = \begin{cases} x, & \text{if } x > 0 \\ \alpha x, & x < 0 \end{cases}$$

Multiply

$\alpha = 0.01$ Multiply

Instead of giving 0 for negative inputs, it gives a tiny negative value (like 0.01x input) This way, the neurons is never completely dead.



↳ Advantages

- Fixes "Dead Neuron" problem or Prevents dying ReLU.
- Allow small negative gradient
- Better gradient flow
- Works well in Deep Networks

↳ Disadvantages

- Negative slope must be chosen Manually.
- Slightly more computation than ReLU

↳ Uses cases

- Deep Neural Network
- CNNs

↳ Used In

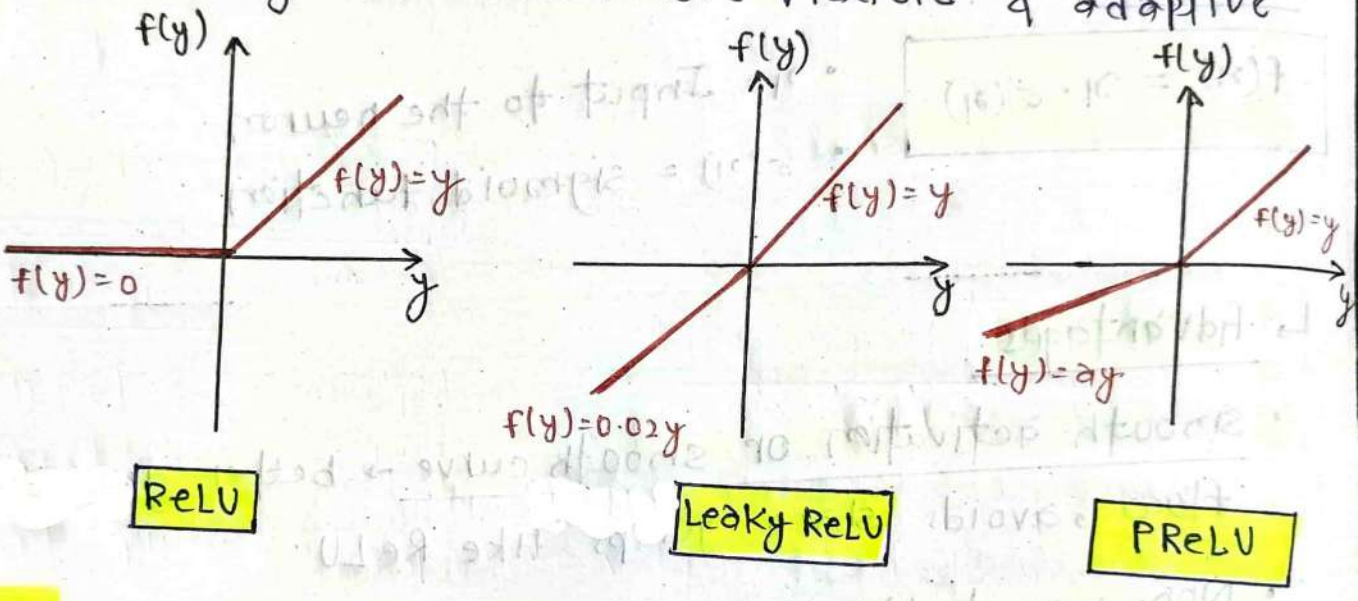
- Hidden layers

→ (6.) PReLU (Parametric ReLU)

It is an improved version of Leaky ReLU.

In Leaky ReLU, the slope for negative value (like 0.01) is fixed by us.

But in PReLU, that slope is learned automatically by model during training. This makes it more flexible & adaptive



Note :-

ReLU = Negative values are killed (output = 0)

Leaky ReLU = Negative values are given a tiny leak (example = 0.01x)

PReLU = Instead of fixing the leak, the model says "I'll learn the best leak slope myself"

↳ Advantages of PReLU

- fixes dead neurons (like leaky ReLU)
- Adaptive - slope is learned, not fixed
- Better accuracy - often improves CNNs & deep networks

↳ **Disadvantages of PReLU**

- extra parameters - slope adds more trainable values
- Risk of overfitting if dataset is small.
- slightly more complex than plain ReLU.

→ **(7) Swish Activation Function**

It is a smooth activation function introduced by Google Researchers.

Formula

$$f(x) = x \cdot \sigma(x)$$

- x = Input to the neuron
- $\sigma(x)$ = sigmoid function

↳ **Advantages**

- **Smooth activation or smooth curve** → better gradient flow, avoids sharp jumps like ReLU.
- **Non-monotonic** can adapt better to complex pattern.
- work well in deep networks (often improves accuracy over ReLU)

↳ **Disadvantages**

- More computationally expensive than ReLU.
- slower training

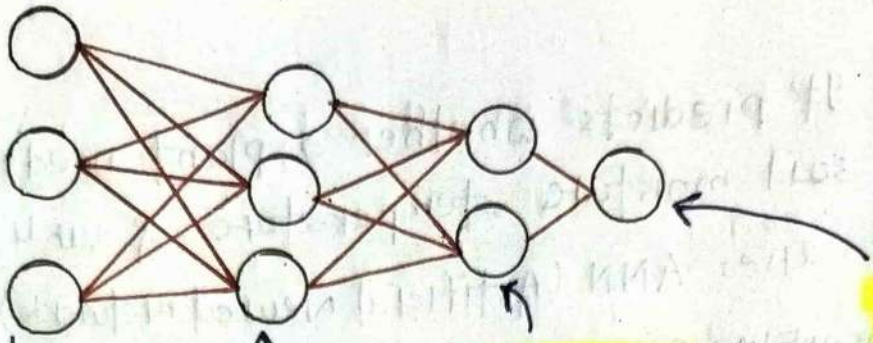
↳ **Use cases**

- Deep Neural Network
- efficient Net
- some modern CNN architecture

↳ **Used In**

- Hidden Layers

Usage of Activation Function



Layer-1 (Input layer)

Layer-2 (Hidden layer)

Layer-3 (hidden layer)

↳ **Input Layer** : It only receives the raw input features & passes them to the first hidden layer

- Activation Function - None

↳ **Hidden Layer**: here activation funtions introduce non linearity, helping network learn complex pattern while reducing the vanishing gradient problem

- **Activation function**
  - * ReLU
  - * PReLU
  - * GELU
  - * Leaky ReLU
  - * swish

↳ **Output Layer**

The activation function depends on the type of problem

- Activation function : Linear (Regression)
- Activation Function : sigmoid (Binary classification)

PROJECT: 1
(SMART IRRIGATION SYSTEM USING DEEP LEARNING)

↳ **Overview**

It predicts whether a plant needs water based on soil moisture, temperature & sunlight hours. It uses ANN (Artificial Neural Network) built with TensorFlow & Keras to perform binary classification, helping automate irrigation decisions & promote efficient water usage.

1. IMPORT LIBRARIES

In [1]:

```
import numpy as np
import tensorflow as tf
# Import tensorflow library for building & training
  deep learning Model
from tensorflow import keras
# Import Keras, TensorFlow's high-level API, to easily
  create neural Network
from sklearn.model_selection import train_test_split
from tensorflow.keras import layers
# Import neural Network layers (such as Input & Dense)
  used to build the model.
import pandas as pd
```

2. CREATE DATASET

In [2]:

```
df = pd.DataFrame ({
    "soil_moisture": [0.10, 0.15, 0.20, 0.25, 0.40, 0.60, 0.35,
                      0.18, 0.45, 0.05, 0.80, 0.27, 0.55,
                      0.70, 0.12, 0.30],
    "temperature_c": [34, 30, 26, 22, 28, 30, 19, 22, 35,
                      24, 33, 33, 21, 25, 20, 29],
    "sunlight_hours": [9, 8, 7, 4, 8, 10, 3, 10, 12, 5, 9,
                       11, 2, 6, 19]
    "needs_water": [1, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0,
                    1, 1]
})

# Create a DataFrame containing the plant dataset
df
df.columns
```

3. SEPRATE FEATURES & TARGET

In [3]

```
X = df [['soil_moisture', 'temperature_c', 'sunlight_hours']]
# select the input Features
y = df ['needs_water']
# select the target variable
```

4. FEATURE SCALING (MIN-MAX NORMALIZATION

In [4]:

```
X_min = X.min()
# finds the minimum value of each feature
X_max = X.max()
# finds the maximum value of each feature
X_scaled = (X - X_min) / (X_max - X_min + 1e-8)
# scales all features to the range [0,1].
X_scaled
# Display the Normalized feature values
```

5. TRAIN TEST SPLIT

In [5]

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size = 0.26, random_state = 42,
    stratify = y)
```

6. BUILD NEURAL NETWORK MODEL

In [6]

```
model = keras.Sequential([
# create a sequential neural Network
        layers.Input(shape = (X_train.shape[1])),
        # Defines Input layer
        layers.Dense(8, activation = 'relu'),
        #Hidden layer with 8 neurons using relu activation.
        layers.Dense(1, activation = 'sigmoid')
        #Output layer with 1 neuron using sigmoid activation
        ])
```

7. COMPILE THE MODEL

In [7]:

```
model.compile(
# configure the model before training
        optimizer = 'adam',
        # optimization algorithm
        loss = 'binary_crossentropy', # loss function
        metrics = ['accuracy']
        # Evaluation Metrics
        )
```

8. TRAIN THE MODEL

In [8]:

```
history_full = model.fit(X_train, y_train, epochs = 100,
        batch_size = len(X_train), verbose = 1)
# Train the Model using full batch gradient descent
  (uses the entire training dataset in one weight update)

history = model.fit(X_train.values, y_train.values,
        validation_data = (X_test.values, y_test.values),
        epochs = 100, batch_size = 1, verbose = 1)
# Train the model using SGD (stochastic gradient Descent)
  (uses one training sample per weight update)

history_minibatch = model.fit(X_train, y_train,
        epochs = 100, batch_size = 100, verbose = 1)
# Trains the Model using Mini Batch gradient descent
  (uses a batch of 100 samples per weight update)
```

9. CONFIGURE THE OPTIMIZER

In [9]:

```
from tensorflow.keras import optimizers
# Import different optimization algorithms
opt = optimizers.SGD (learning-rate=0.01 , momentum=0.9)
# creates an SGD optimizer with a learning rate of 0.01
  & momentum of 0.9
```

* **Verbose** - It controls how much training information is displayed during model training.

- verbose=0 → No output is shown.
- verbose =1 → shows a progress bar & training metrics for each epoch.
- verbose =2 → shows one summary line per epoch (no progress bar)

→ Training & Validation Metrics

(1.) Loss

- Measure how wrong the model's predictions are.
- compute using the chosen loss function.
- Lower is Better
- The optimizer (ex- SGD, Adam) updates the weights to minimize this loss.

(2) Accuracy

- Percentage of correctly classified training samples.
- Higher is better.
- Accuracy alone can be misleading for imbalanced dataset, so other metrics (Precision, Recall, F1 score, ROC-AUC) may also be needed.

(3) Val_loss

- Loss on the unseen validation dataset.
- shows how well the model generalizes.
- Lower is Better
- This is often the most important metric to monitor.
- If loss decreases but val_loss increases, the model is likely overfitting.

(4) val_accuracy

- Accuracy on the unseen validation dataset.
- Higher is Better.
- If accuracy is high but val-accuracy is much lower, the model is likely overfitting.

↳ **Where to pay attention**

- loss & accuracy = Performance on the training dataset.
- Val_loss & val_accuracy = Performance on unseen validation data.

→ **Loss function**

It Measures how far the model's prediction ($\hat{y}$) is from the actual value (y).
It provides the error used during backward propagation so the optimizer can update the weights & reduce the error

↳ **Loss function Vs Cost Function**

If a dataset has 100 samples:

- **Loss Function** → Error for one sample
- **Cost Function** → Average error of all 100 samples.

→ **Types of Loss Functions**

(1) Regression

Predict continuous numerical values.
ex - House prices, Temperature, Salary, stocks etc.

↳ Types

(1) MSE (Mean squared error)
(2) MAE (Mean Absolute error)
(3) Huber loss
' MSLE

(2) classification

Predict categories or classes.
ex - spam Detection, Review, Disease Prediction

↳ Types

(1) Binary class entropy
(2) classification class entropy

→ Regression Loss Functions

↳ Mean squared Error (MSE)

It calculates the average squared difference between the predicted value & the actual value.

• Formula

$$MSE = \frac{1}{n}\sum (y - \hat{y})^2$$

- n = no. of samples
- y = Actual value
- $\hat{y}$ = Predicted value

• Advantages

* Easy to calculate & differentiate
* Penalizes large errors heavily because errors are squared.
* works well with gradient descent.
* Most commonly used regression loss

• Disadvantages

* Highly sensitive to outliers
* Large prediction errors can Dominate training.

↳ Mean Absolute Error (MAE)

It calculates the Average absolute difference between predicted & Actual values.

• Formula

$$MAE = \frac{1}{n}\sum |y - \hat{y}|$$

• Advantages

* Robust to outliers.
* Every error contributes equally
* Easy to understand

• Disadvantages

* Does not heavily penalizes large errors.
* Training can be slower because gradient are less smooth

↳ Huber loss

It combines the advantages of MSE & MAE.

- Small error → MSE (mean squared error)
- Large errors → MAE (mean Absolute error)

This provides stable learning while reducing the effect of outliers.

• Advantages

* Less sensitive to outliers than MSE
* More stable than MAE
* Good balance between accuracy and robustness

• Disadvantages

* Requires choosing a threshold.
* slightly more complex than MSE or MAE

• Best Use

Dataset that contains some outliers but still require accurate predictions.

↳ Mean squared Lograrithmic Error (MSLE)

It calculates the squared difference after applying a logarithm to both predicted & actual values.

• Formula

$$MSLE = \frac{1}{n}\Sigma(\log(y+1) - \log(\hat{y}+1))^2$$

• Best Use

* Population growth
* Revenue growth
* Financial forecasting
* Sales growth prediction

• Advantages

* good for exponential growth prediction
* Reduces the impact of very large values.

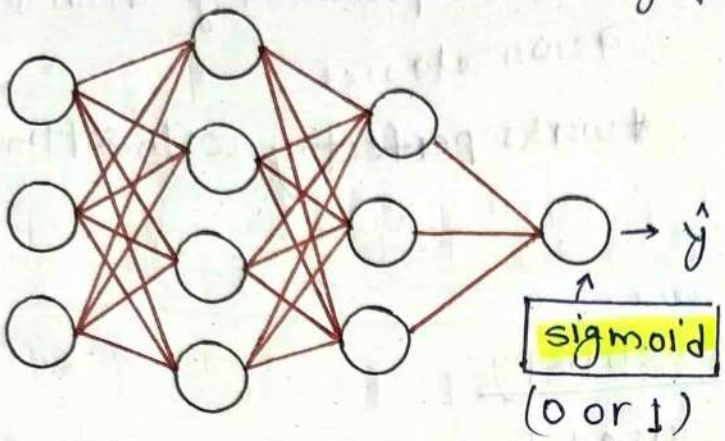
• Disadvantage

* Not suitable for normal regression Problems
* cannot handle -ve values.

→ Classification Loss Functions

↳ Binary Cross Entropy (Log loss)

Used for binary classification, where there are only two possible classes.



- output Activation = sigmoid

Ex-

Animals
0
1
0
⋮
y

output will be

- **Used In**
  - * Yes/NO
  - * Fraud / Not fraud etc.

- **Advantages**
  - * Best choice for binary classification.
  - * Produces probability outputs
  - * works efficiently with sigmoid activation.
- **Disadvantages**
  - * cannot be used for multi-class classification

↳ categorical cross Entropy

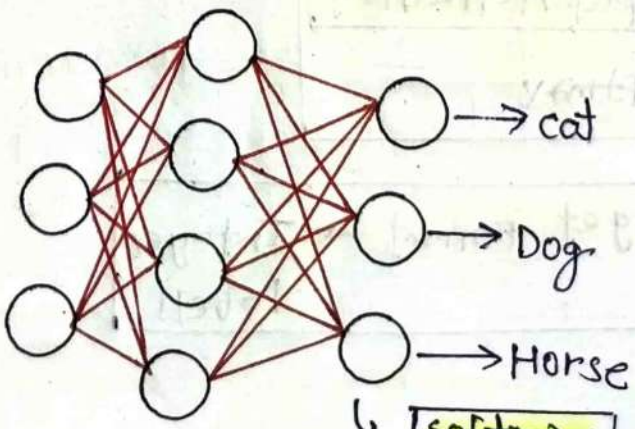
Used for multi-class classification when the target labels are one-hot encoding.

ex-

Animal
cat
Dog
horse
cat
horse
dog

one hot → encoding

Cat	Dog	horse
1	0	0
0	1	0
0	0	1
1	0	0
0	0	1
0	1	0



↳ softmax

Output Activation

Softmax

• Target format → one-hot encoding

• Disadvantages

* Requires one-hot encoding
* Uses more memory for many classes.

• Advantages

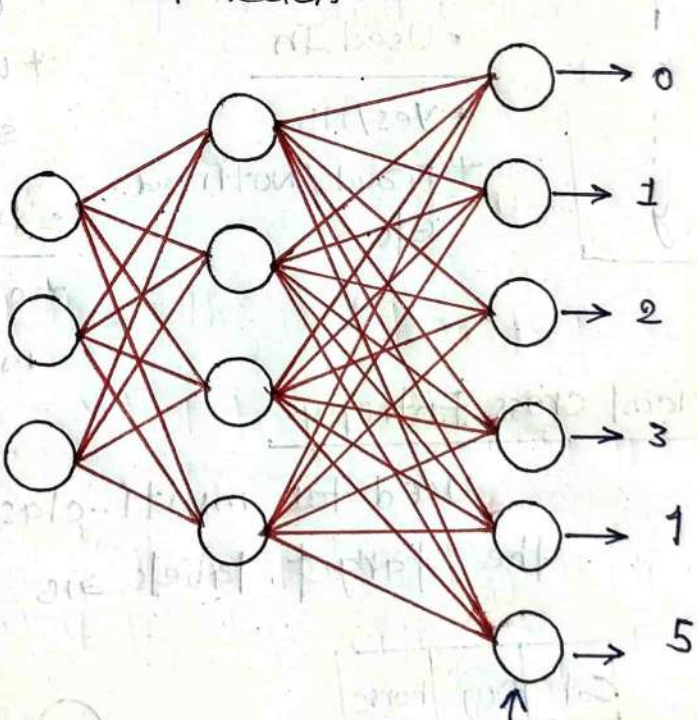
* Best for multiclass classification
* Produces probability distribution across classes
* Works perfectly with softmax.

↳ Sparse categorical cross-entropy

Used for multiclass classification when target labels are stored as integers instead of one-hot encoded vectors.

Ex-

Animal
0
1
2
1
3
4
5
1
2



• Output Activation

Softmax

• Target Format → Integer labels

- Advantages

* No need for one-hot encoding
* saves memory
* Faster preprocessing
* ideal for datasets with many classes

- Disadvantages

* only works with integer labels.

→ Choosing the Right Loss Function

Problem Type	Loss Function	output Activation
Regression	Mean squared Error (MSE)	Linear
Regression	Mean Absolute error (MAE)	Linear
Regression	Huber loss	Linear
Regression	Mean squared Logrithmic error (MSLE)	Linear
Binary classification	Binary cross Entropy	sigmoid
Multi-class (one-hot Labels)	categorical cross-Entropy	softmax
Multi-class (Integer Labels)	Sparse categorical cross entropy	softmax

→ optimizers

It is the algorithm responsible for updating the weights & biases of a neural network to minimize the loss.

↳ working of optimizers

(1) Make predictions using current weights
(2) calculate the loss (error).
(3) Compute gradients using backpropagation.
(4) Update weights in the direction that reduces the loss.
(5) Repeat for many epochs until the model converges.

↳ During training

(1) Forward Propagation - The model predicts an output.
(2) Loss Function - calculates how wrong the prediction is
(3) Backward propagation - computes the gradients (partial derivatives) of the loss with respect to each weight
(4) optimizer - Uses these gradients to update the weights & biases, reducing the loss.

→ Global Minimum Vs Local Minimum

Global Minimum	Local Minimum
• The lowest point possible on the loss curve	• A low point that is not the lowest overall.
• Produces the minimum loss.	• The optimizer may get stuck here
• Represents the best solution	• Produces higher loss than global minimum.

Optimizers Goal = Reach the global minimum instead of getting trapped in local minimum

→ Gradient Descent

It is foundation of most optimization Algorithms.

It updates the weights by moving in the opposite direction of the gradient, since the gradient points towards increasing loss.

Weight update Formula

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

- W = weight
- η (eta) = Learning Rate
- L = loss function
- $\partial L / \partial W$ = Gradient of loss

* Learning Rate Determines how large each weight update is.

→ Types of Gradient Descent

The main difference between these all methods is how much data is used before updating the weights.

↳ (1.) Batch Gradient Descent (Vanilla Gradient Descent)

It uses entire training dataset to calculate the gradients before updating weights.

Ex- Dataset = 100,000 samples
weight updates per epoch = 1

• advantages

* smooth & stable updates (less noisy).
* converges steadily because gradient

• disadvantages

* slow when dataset is huge
* Requires a lot of RAM/VRAM to load the full dataset at once.

• Used In - Small dataset, Academic learning, Simple ML models

↳ (2.) Stochastic Gradient Descent (SGD)

It updates the weights after every single training sample.

ex- Dataset = 100,000 samples
weight updates per epoch = 100,000

• Advantages

* starts learning immediately
* Requires very little memory
* suitable for very large dataset.
* Faster updates.

• Disadvantages

* Noisy & unstable learning path.
* Loss fluctuates frequently
* can overshoot the optimum
* May get stuck in local minima

• Best Used

* very large datasets
* online learning
* streaming data

↳ Mini-Batch Gradient Descent

* Divides the dataset into small batches & updates the weights after processing each batch.

Ex Dataset = 100,000 samples
Batch size = 32
weight updates per epoch ≈3125

• Used In

Almost every modern deep learning applicatin

• Advantages

* Faster than Batch Gradient Descent
* More stable than SGD
* efficient memory usage.
* supports GPU acceleration.
* Most widely used in Deep learning

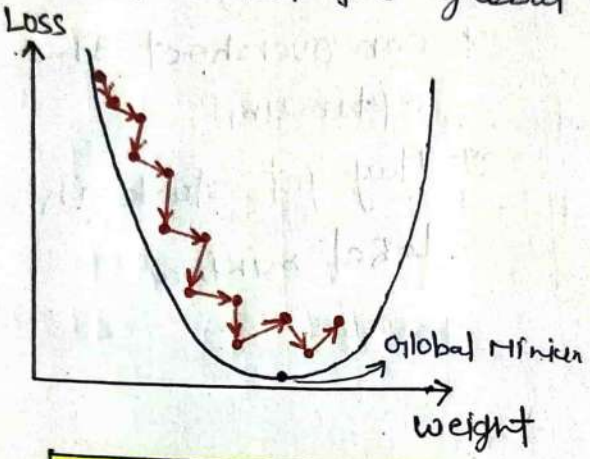
• Disadvantages

* Requires selecting an appropriate batch size.
* Too small → Noisy updates
* Too large → higher memory usage

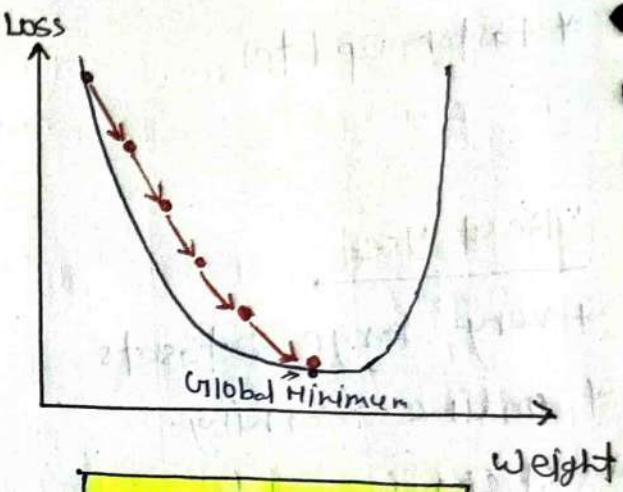
→ Momentum

It is an optimization technique that uses previous weight updates to speed test up learning & reduce oscillations, helping the model reach the global minimum faster.

↳



Without Momentum (SGD)



With Momentum

- without momentum → The ball slows down at every bump
- with momentum → The ball gains speed & rolls over small bumps, continuing towards the bottom

* 0.9 momentum commonly used.

→ AdaGrad (Adaptive Gradient Descent)

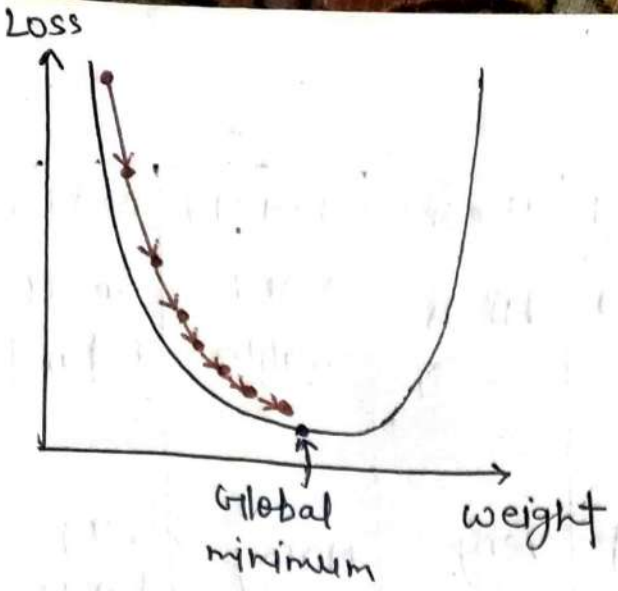
It is an optimizer that automatically adjusts the learning rate for each parameter during training.

↳ Advantages

- Adaptive learning rate
- Less manual tuning

↳ Disadvantages

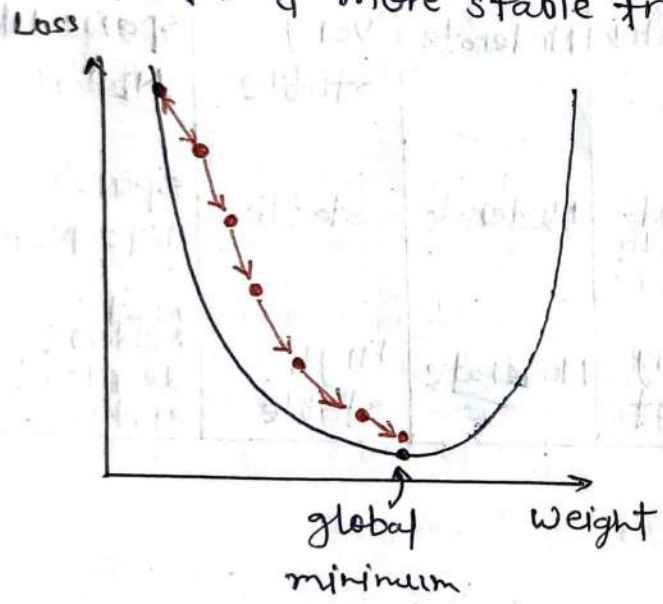
- Learning rate can become too small, slowing or stopping learning



- Larger steps at beginning
- steps become smaller over time

→ **Adam optimizer (Adaptive moment estimator)**

It is an optimizer that combines momentum & learning rate (AdaGrad like) for faster & more stable training.



- Fast descent
- smooth path (with momentum)
- Quickly reaches minimum
- Uses an adaptive learning rate for each parameter.

↳ **Advantages**

- Fast convergence
- stable training
- Works well for most deep learning models

↳ **Disadvantages**

- Uses more memory than SGD
- can sometimes generalize slightly worse than SGD

* Most widely used optimizer in deep learning.

→ Comparision Table

	Weight update	speed	Memory	Stability	Best for
Batch Gradient Descent	Entire Dataset	slow	High	very stable	Small Dataset
Stochastic Gradient (SGD) Descent	One sample	Fast	Very low	Noisy	very large dataset
Mini-Batch Gradient Descent	Small batches	Fast	Moderate	stable	Most deep learning Models
Momentum	Gradient + Previous velocity	Faster	Moderate	very stable	Deep networks
AdaGrad (optimizer)	Adaptive learning rate	Mod-rate	Moderate	stable	sparse data, NLP
Adam (optimizer)	Momentum + Adaptive learning rate	very Fast	Moderate	High stable	Most Modern deep learning application

PROJECT:2
IRIS PROJECT USING ANN

→ Overview

- classify Iris flowers into three species.
- Use the Iri's dataset for classification.
- Compares Perceptron Model & ANN (Artifical Neural Network) Model.
- Apply data preprocessing (encoding & feature scaling)
- Evaluate models using accuracy, classification report, & confusion matrix.
- Identify the model with the best classification performance.

1. IMPORT LIBRARIES

In [1]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as pt
import seaborn as sns
import warings
warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,
StandardScaler
```

```
from sklearn.linear_model import Perceptron
# Import Perceptron (Single Layer neural Network)
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix
#Import Model evaluation Metrics
import tensorflow as tf
# Import tensorflow library
from tensorflow.keras.models import sequential
# Import sequential model, help us to build a
  neural Network layer-by-layer in a Linear fashion.
from tensorflow.keras.layers import Dense
# Import Dense layer, Fully connected layer (every
  neuron connects to every neuron in the next year)
from tensorflow.keras layers import Dropout
# Import Dropout layer, Randomly drops some neurons
  during training to prevent overfitting.
from tensorflow.keras.utils imports categorical
# Import one-hot encoding function

'''LOAD DATASET'''

df = pd.read_csv ('Iris.csv')

df.read()
```

2. EXPLORE DATASET & FEATURE SELECTION

In [2]:

```
df ['species'].value_counts()
#count samples in each species
df.info()

sns.pairplot(df, hue='species')
#visualize relationships between features
X = df.drop(columns=['species', 'Id'], axis=1)
#select input features
y = df['species']
#select target variables
```

3. ENCODE TARGET VARIABLES & TRAIN-TEST-SPLIT

In [3]:

```
encoder = LabelEncoder()

y_int = encoder.fit_transform(y)
# convert class labels into integers
y_int

X_train, X_test, y_train, y_test = train_test_split(
        X,
        y_int,
        test_size=0.2,
        random_state=42,
        stratify=y_int)
#split data into training & testing sets
```

4. FEATURE SCALING

In [4]:

```
Scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
# Fit scaler on training data & transform it
X_test_scaled = scaler.transform(X_test)
# Transform testing data using the same
  scaler
```

5. TRAIN PERCEPTRON MODEL & MAKE PREDICTION

In [5]:

```
per = Perceptron(
#create Perceptron classifier
        max_iter = 1000,
        # Maximum no. of times the model learns
          from the training data
        random_state = 42)
        #set a fixed random value for the
          same results every time
per.fit(X_train_scaled, y_train)
#train the model

y_pred_percep = per.predict(X_test_scaled)
# Predict test data
accuracy_score(y_test, y_pred_percep)
print(classification_report(y_test, y_pred_percep))
```

6. CONVERT LABELS TO ONE HOT ENCODING

In [6]:

```
y_train_cat = to_categorical(y_train, num_classes=3)
# convert training labels into ONE HOT ENCODING format
y_test_cat = to_categorical(y_test, num_classes=3)
# convert testing labels into ONE HOT ENCODED format
```

7. BUILD ANN MODEL & COMPILE

In [7]:

```
model = Sequential([
# create ANN Model
        Dense(16, input_dim=4, activation='relu'),
        #create Hidden layer with 16 neurons, no. of
         input feature is 4, Activation function is relu
        Dense(8, activation='softmax')
        # 2nd Hidden layer with 8 neurons, activation
          function is softmax.
        Dense(3, activation='softmax')])
model.compile(
#configure the Model for training
        optimizer='adam',
        #Adam as optimizer
        loss='categorical_crossentropy',
        #Loss function for multiclass classification
        metrics=['accuracy'])
        #performance evaluation metrics
```

8. TRAIN ANN MODEL & EVALUATE

In[8]:

```
history = model.fit(X_train_scaled, y_train_cat,
#Train Neural Network
          epochs = 100,
          #No: of complete training cycles
          batch_size = 8,
          #No. of samples in each batch
          validation_split = 0.2,
          #use 20% of training data for validation
          verbose = 1 )
          #Display training progress

loss, acc = model.evaluate(X_test_scaled, y_test_cat,
            verbose = 1)
#Evaluate the model using test data

Print(acc)
# Display the test accuracy
```

9. PLOT TRAINING & VALIDATION ACCURACY

In[9]:

```
plt.figure(figsize = (10,4))
plt.plot(history.history['accuracy'], label = "train Acc")
# Plot training Accuracy
plt.plot(history.history['val_accuracy'], label = "val Acc")
# Plot valuation accuracy
```

→ White box model

These models are transparent & easy to interpret. we can clearly understand how the model reaches its final decision.

Ex- • Decision Tree

* every decision is visible
* we can trace each split based on measures like - Gini Index or Entropy
* Easy to explain why a prediction was made.

• Linear Regression

→ Black Box Models

These models are complex & difficult to interpret. we can know the inputs & outputs, But understanding the exact internal decision-making process is very difficult.

Ex. • Random Forest

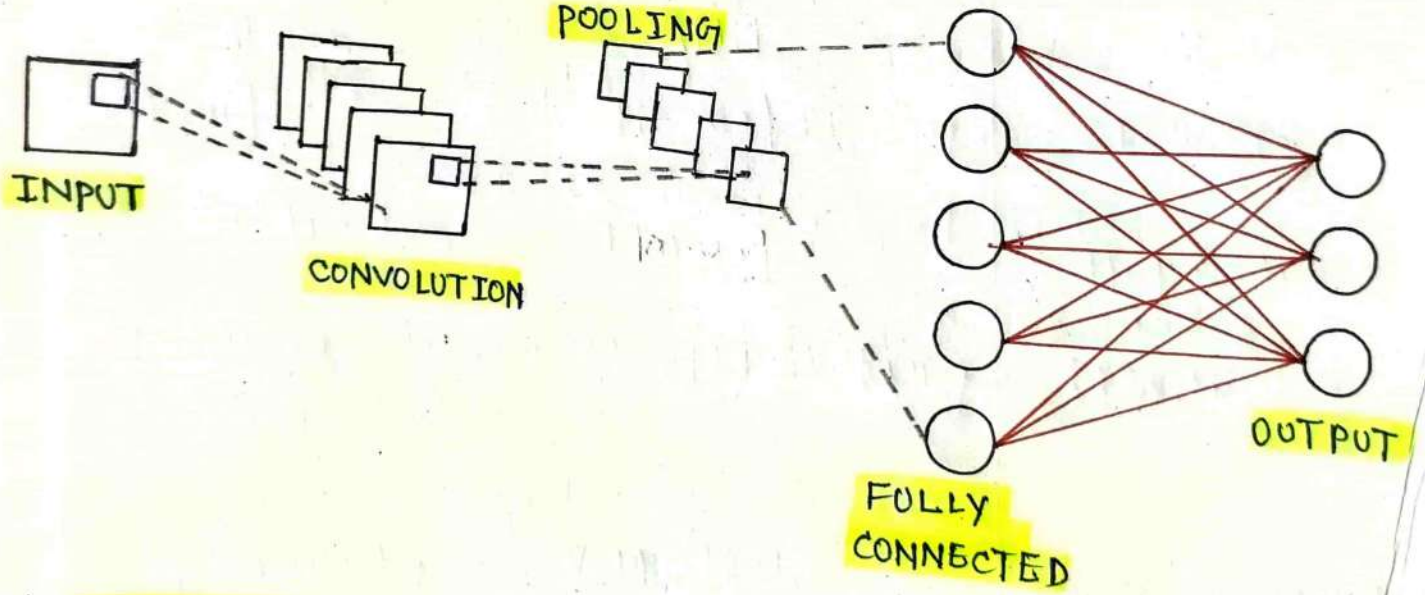
* Built from hundreds or thousands of Decision Tree.
* Although each tree is interpretable, the combined decision of all trees makes the overall model difficult to explain.

• ANN / Deep Learning

PART - II
CNN (CONVOLUTIONAL NEURAL NETWORK)

→ **CNN (convolutional Neural Network) :**

It is a type of deep learning neural network designed for processing images, videos & other grid like data. It automatically learns important features such as edges, shapes & textures using convolution & pooling layers, making it highly effective for image recognition & classification tasks.



↳ **Advantages**

- Automatic feature extraction
- Preserves spatial information
- Fewer parameters than ANN
- Higher accuracy for image tasks
- Reduces overfitting
- Recognizes shifted objects

↳ **Disadvantages**

- Requires large dataset
- High training time
- computationally expensive
- Need powerful hardware (GPU or TPU)
- Difficult to interpret

↳ Uses of CNN

- Image classification
- object detection
- Face recognition
- Medical image analysis
- self driving cars
- OCR (optical character Recognition)
- video analysis
- Image segmentation
- security & surveillance.

→ Why CNN instead of ANN for Images ?

ANN (Artifical Neural Network) are not efficient for images data because images contain a huge number of pixels.

Ex- ↳ a 32 X 32 pixels RGB image

- Pixels = 32 * 32 X 3 = 3072 inputs
  (3 ↑ Red, Green, Blue)

↳ a 1920X1080 RGB image

- Pixels = 1920 X 1080 X 3 = 6220800 inputs
  (3 ↑ Red, green, blue)

every input neuron connects to every neuron in the next layer, creating millions or billions of weights.

↳ Problems with ANN

(1) High computational Cost

- every pixel becomes an input neuron.
- Requires enormous memory & computation.
- Training becomes very low.

(2.) Loss of spatial Information

Images are Naturally 2D, After flattening, the model no longer knows which pixels were neighbors. in short it forgets the arrangement of pixels.

(3) overfitting

since ANN has a huge no. of parameters. it often;

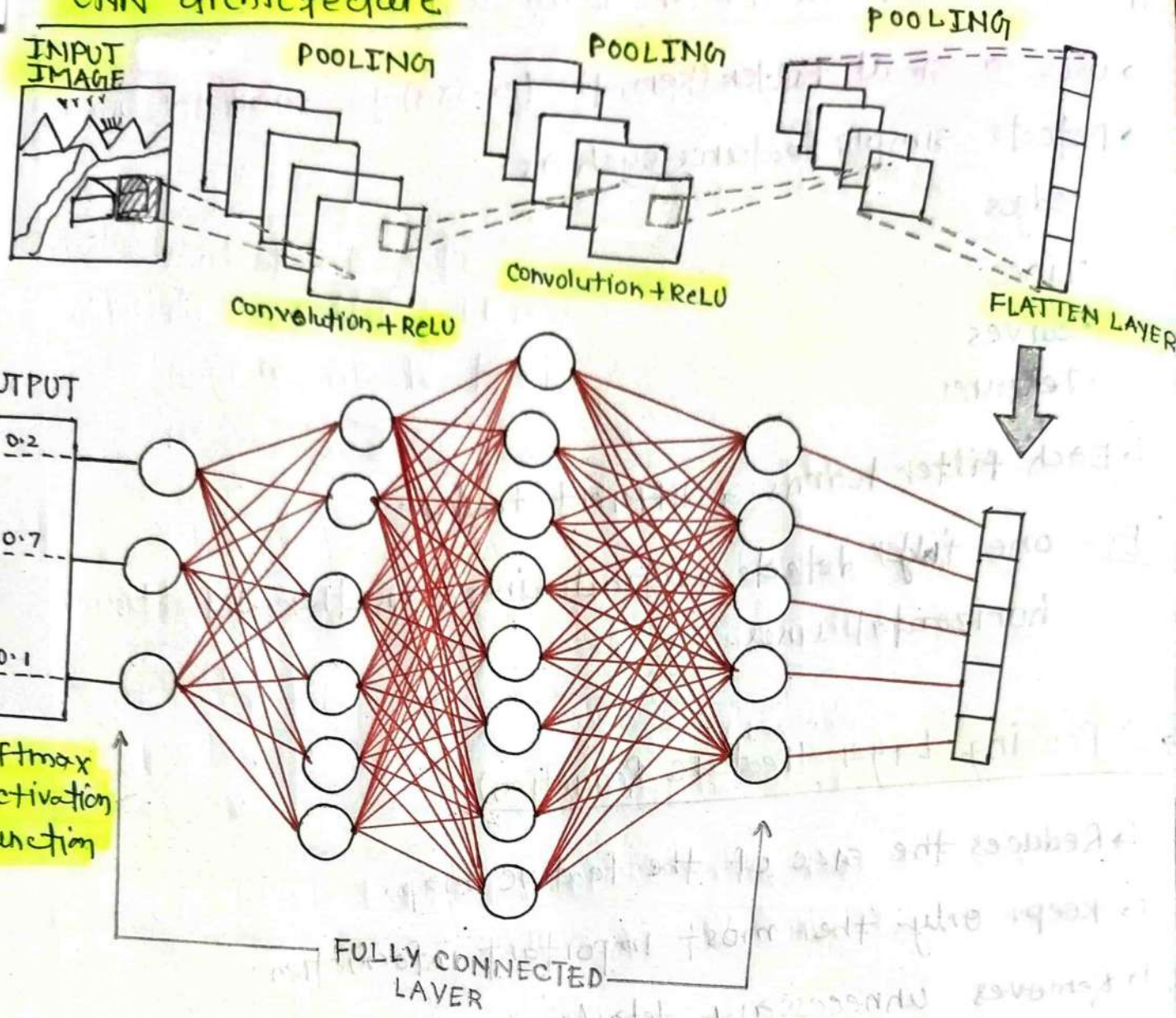
- memorizes training Images
- performs poorly on new images.

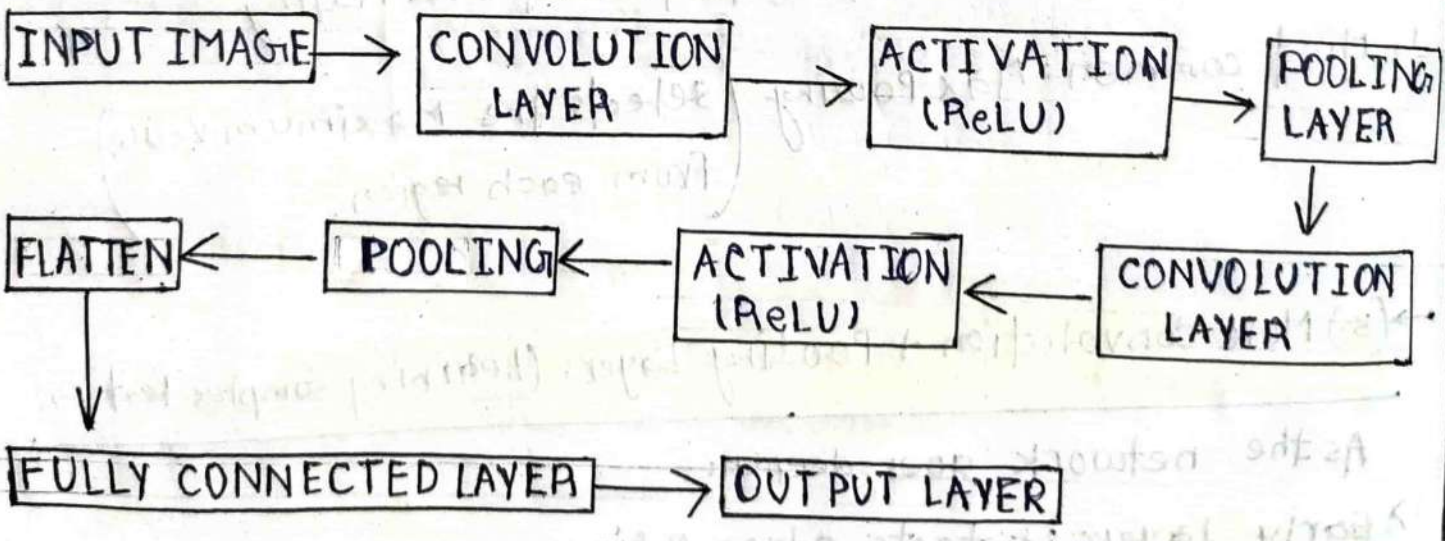
This is called overfitting.

→ ## ANN Vs CNN

Feature	ANN	CNN
Input	1D vector	2D images
Feature Extraction	Manual	Automatic
spatial information (Location of pixel in image like	Lost after flattening	Preserved
No. of Parameters	very High	Lower (shared filters)
Computational cost	High	Lower than ANN for images
Best Used for	Tabular & numerical data	Images, videos & computer vision
Image Recognition Accuracy	Lower	Higher
overfitting	More prone	Less prone

CNN architecture





(1.) Convolution Layer (Feature Extraction)

↳ Uses a small filter (kernal) (ex-3x3) to scan the image

↳ Detects simple features such as:

- edges
- Lines
- curves
- Textures

It is a detective searching for clues in different part of an image.

↳ Each filter learns a different feature.

Ex- one filter detects vertical lines, another detects horizontal lines.

(2) Pooling Layer (Feature Reduction)

↳ Reduces the size of the feature map.

↳ keeps only the most important information.

↳ Removes unnecessary details & noise.

↳ Reduces computation & helps prevent overfitting

↳ Most common: Max Pooling (selects the Maximum value from each region)

(3.) More convolution + Pooling Layers (Learning Complex features)

As the network goes deeper:

↳ Early layers: Detects edges & lines.

↳ Middle layers: Detect shapes & object parts.

↳ **Deep Layers**: Recognize complete objects.

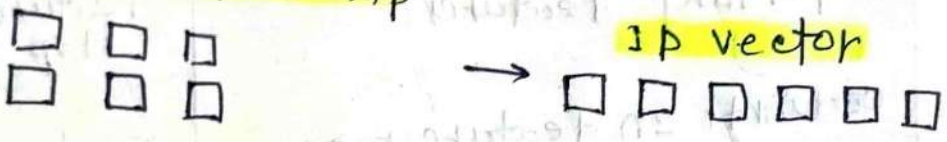
Edges → shapes → object parts → complete object

→ (4.) Flatten Layer

↳ convert the 2D feature maps into a 1D vector.

↳ This vector becomes the input to the fully connected layer.

Ex - 2D Feature Map → 1D vector



→ (5.) Fully Connected Layer (classification)

↳ Works like a normal ANN.

↳ Uses all extracted features to predict the final class.

↳ Produces the probability for each class.

ex - cat → 90%.
Dog → 20%.
Rabbit → 10%.

The class with the highest probability is the final prediction.

→ short : CNN layers & Their Functions

Layers	Main Function	Output
Input Layer	Receives the image	Image
Convolution Layer	Extracts features using filter	Feature Maps
ReLU Activation	Removes negative value & adds non-linearity	Activated feature Map
Pooling Layer	Reduces dimensions & keeps important features	smaller Feature Map
Flatten Layer	converts 2D feature map into a 1D vector	Feature vector
Fully connected Layer	Combines learned features for classification	Scores
output Layer (softmax/sigmoid)	Predicts the final class probabilities	Final Prediction

Finding Edges in CNN

convolution layer is the first layer of a CNN. It detects basic features such as edges, lines, curves & textures, which are later combined to recognize complex objects like - faces, cars or animals.

The convolution layer uses several steps to Edges Finding -

(1) Pixel Matrix (For Examples it can use More)

A computer stores an image as matrix of pixel values.

For greyscale image:

- 0 = Black
- 255 = White
- Value between 0-255 represent different shades of grey.

Ex - (6X6 Image) greyscale

0	0	0	0	0	0
0	255	255	255	255	0
0	255	255	255	255	0
0	255	255	255	255	0
0	255	255	255	255	0
0	0	0	0	0	0

For RGB

- Red = (255,0,0)
- Blue = (0,0,255)
- Green = (0,255,0)

(2) Filter (Kernal)

It is a small matrix (usually 3x3 or 5x5) that slides across the image.

Ex- for a vertical edge filter

-1	-1	-1
0	0	0
1	1	1

* **Different filter detect**

- Vertical edges
- Horizontal edges
- Diagonal edges
- Curves
- Textures

(3) Convolution operation

The filter slides across the image.

At each position -

(i) Multiply corresponding pixel values & filter values

(ii) Add all the results

(iii) store the sum in the output.

This process is repeated over the entire images.

- output = Feature Map

(4) Edge Detection

When the filter passes over an area with a strong change in pixel values (ex- black to white), the output is large value, indicating an edge.

When the filter passes over a uniform region, positive & negative values cancel each other, producing a value close to 0.

High value → Edge Detected

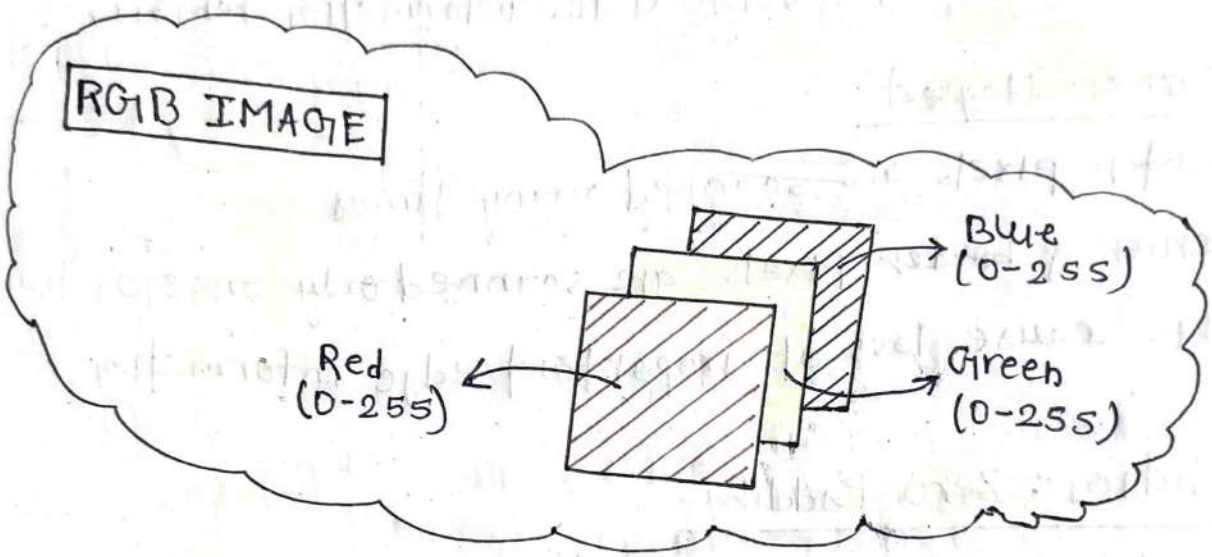
low value → No Edge

- output = Feature Map

→ (5) output size

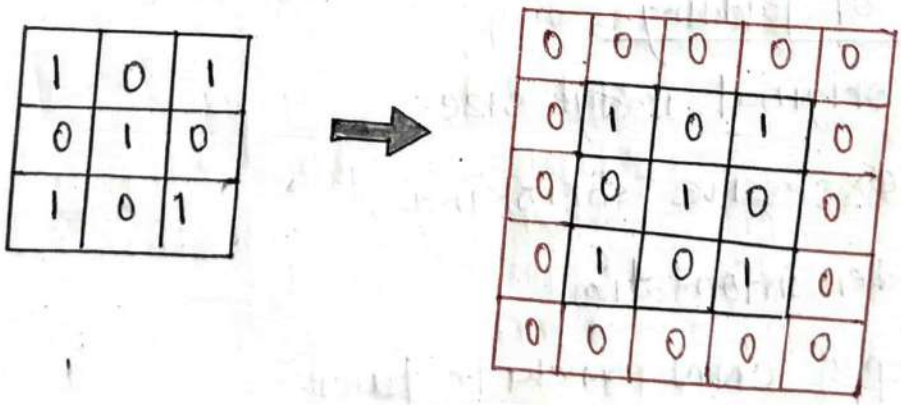
Ex:
- Input Image = 6X6
- Filter = 3X3
- output Feature Map = 4X4

for RGB images, the filter becomes 3X3X3 because image has three channels (R,G,B)



→ Padding

It means adding a border of zeros (dummy pixels) around the image before convolution.



↳ Problems without Padding

(i) Shrinking Dimensions

every convolution reduces image size.

Ex- Input 5X5 Matrix Image → 3X3 Filter → output: 3X3

- If Many convolution layers are used, the image keeps shrinking until very little information remain

(ii) Corner Neglect

- center pixels are scanned many times.
- corner & border pixels are scanned only once or twice
- This cause loss of important edge information

↳ solution : Zero Padding

Add zeros around the image

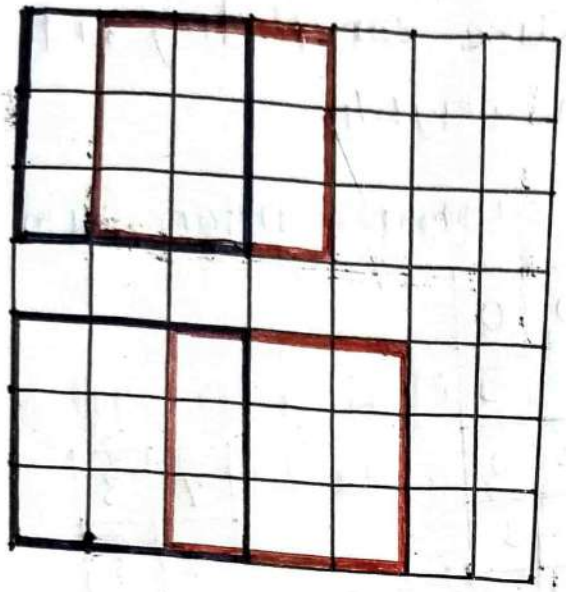
ex- original image 5X5 → After Padding 7X7 → 3X3 Filter → output 5X5

↳ Advantages of padding

- Preserve original image size.
- Prevents excessive shrinking.
- Keeps border information.
- Allows deeper CNN architectures.

→ stride

It is the no. of pixels the filter moves after each convolution.



↳ stride = 1

- Filter moves 1 pixel at a time.
- Produces detailed feature maps.
- Higher computation

↳ stride = 2

- Filter moves 2 pixels at a time.
- Produces smaller feature maps.
- Faster computation.

↳ Use of stride

- Reduces computation
- Reduces memory usage
- speeds up training
- Produces smaller features maps.

Note- computer in (2011-12) had limited computational power, so higher strides were commonly used to process image efficiently.

But today "stride = 1" is preferred because modern hardware can process larger images while preserving more detail.

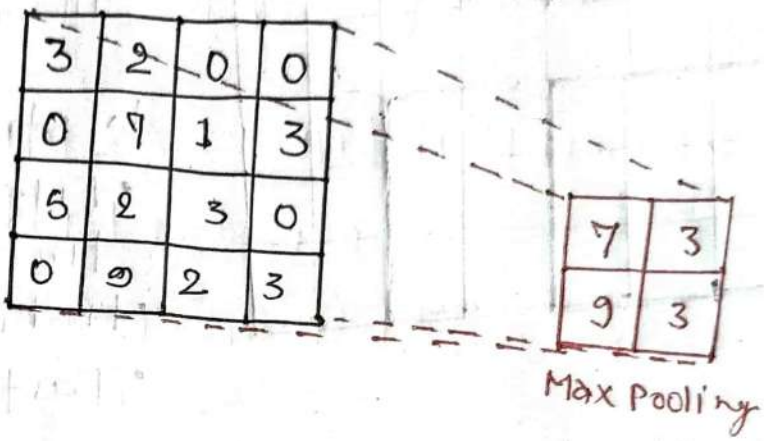
Higher strides are still used when only rough, low level features are needed.

→ Pooling Layer

It is a down sampling operation that reduces the size of feature maps.

Unlike convolating, it doesn't use filter or learn weights.

It summaries information from small regions (eg - 2X2 window) of the feature Map



↳ Pooling Types

(1) Max Pooling (Most commonly)

It selects the largest (max) value of each pooling window (usually 2X2).

Ex - Input = | 2 | 6 |
| --- | --- |
| 4 | 8 |

Output = 8

- **Advantages**
  - *Keep the strongest feature.
  - *Reduces computation & Memory
  - *Preserves important edges, texture & remove noise
- **Disadvantages**
  - *Discard some information
  - *May ignore small features.

(2) Avg pooling

Calculate the average of all values in the pooling window.

Ex - Input = | 2 | 6 |
| --- | --- |
| 4 | 8 |

Output = 5

- **Advantages**
  - *Produces smoother feature maps.
  - *Retains overall information
  - *Less sensitive to noise

(3) Min pooling

selects the smallest (min) value of each pooling window.

Ex - Input = | 2 | 6 |
| --- | --- |
| 4 | 8 |

Output = 2

- **Advantages**
  - *Highlights dark or low intensity regions
  - *Can supress bright background feature
  - Rarely used in CNN

↳ Use of Pooling

(1) Reduces Memory & computation

ex- If suppose convolution layer produces 100 feature Maps.

If each map is 30

then :- 100 X 30 MB = 3000 MB (3 GB)

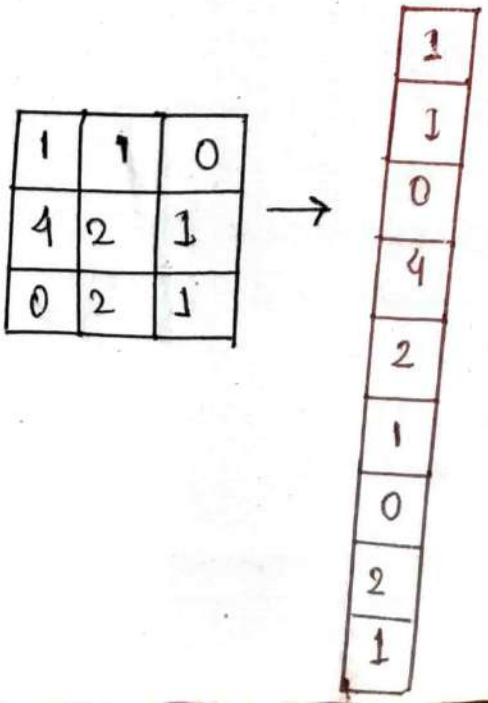
Pooling reduces the size of feature maps, greatly decreasing memory usage & speeding up computation.

(2) Translation Invariance

Pooling helps the model recognize an object even if its position changes.

It focuses on the important feature rather than exact pixel locations, allowing CNN recognize both images as the same object

→ Flatten Layer



It converts the 3D feature maps produced by the convolution & pooling layers into 1D vector

- It allows the output to be passed to the Fully connected (Dense) Layer for final classification.

PROJECT : 1
HANDWRITEEN DIGIT RECOGINITION USING PERCEPTRON, ANN & CNN

Overview

- Built model to classify handwritten digit using the MNIST dataset.
- Implemented Perceptron, ANN (Artifical Neural Network), CNN (Convolutional Neural Network)
- Applied data preprocessing, normalization & one-hot encoding
- Compared model performance using accuracy & validation curves.
- Evaluated results with a confusion matrix & prediction comparism.
- Demonstrated that CNN achieves the highest accuracy for image classification.

1. IMPORT LIBRARIES

In[ ]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
from sklearn.preprocessing import LabelEncoder,
StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
# Import Perceptron used for simple linear classification
  tasks
from sklearn.metrics import accuracy_score,
classification_report, confusion_matrix
from tensorflow.keras.model import sequential
# sequential used to build a neural network layer-by
  layer in keras
from tensorflow.keras.layers import Dense
# Dense make the final predictions
from tensorflow.keras.layers import Conv2D
# It extracts features
from tensorflow.keras.layers import Flatten
# It reshape them 3D to 1D
from tensorflow.keras.layers import MaxPooling2D
# It reduces size
from tensorflow.keras.layers import Dropout
# It prevent overfitting
from tensorflow.keras.utils import to_categorical
```

2. LOAD DATASET & EDA

In [2]:

```
df = pd.read_csv("mnist_train.csv")
df_test = pd.read_csv("mnist_test.csv")
df.head()
df.shape
df.columns
df.info()
df.isnull().sum()
```

3. DATA PREPROCESSING

In [3]:

```
"""SEPERATE FEATURE AND LABELS"""
X_train = df.drop("label", axis=1).values
# Training features
y_train = df["label"].values
# Training labels
X_test = df_test.drop("label", axis=1).values
# Testing features
y_test = df_test["label"].values
# Testing labels

"""NORMALIZE PIXEL VALUES"""
X_train = X_train.astype("float32") /255.0   → Max pixelvalue
# scale traing pixels

X_test = X_test.astype("float32") /255.0
# scale testing pixels
```

```
"""RESHAPE IMAGES"""
X_train_img = X_train.reshape(-1, 28, 28)
#Training image format
```
auto calculate no. of image
height & width of Image

```
X_test_img = X_test.reshape(-1, 28, 28)
#training testing image format

"""ONE-HOT ENCODE LABELS"""
y_train_cat = to_categorical(y_train, 10)
#encode training labels

y_test_cat = to_categorical(y_test, 10)
#Encode testing labels
```
Total output classes (digit 0-9)

4. PERCEPTRON MODEL & COMPILE

In[4]:

```
perceptron = Sequential([
#Perceptron model
    Flatten(input_shape=(28, 28)),
#Flatten image
    Dense(10, activation="softmax")])
#output layer

perceptron.compile(
#compile helps prepare the model for training
    optimizer="sgd"   #optimizer
    loss="categorical_crossentropy"
#Loss function, categorical cross entropy measure
 prediction error
    metrics=["accuracy"] #Evaluation Metric
```
height x width

5. TRAIN & EVALUATE PERCEPTRON

In[5]:

```
history_percp = perceptron.fit(X_train_img, y_train_cat,
        epochs=5,   #epochs = Foward + backward propagation
        batch_size=32
        validation_data=(X_test_img, y_test_cat),
        verbose=1) # show progress

acc_percp = perceptron.evaluate(X_test_img, y_test_cat,
            verbose=0)[1]
                    ↳helps to extract accuracy value
acc_percp
```

6. ANN MODEL & COMPILE

In[6]:

```
ann = sequential([
        Flatten(input_shape=(28,28)),
        #3D to 1D
        Dense(128, activation="relu"),
        #Hidden layer with 128 neurons
        Dense(64, activation="relu"),
        # Hidden layer with 64 neurons
        Dense(10, activation="softmax")])
        #output layer with 10 neurons & activation function is
          soft max

ann.compile(optimizer="adam", loss="categorical_crossentropy",
        metrics=["accuracy"])
```

7. TRAIN & EVALUATE MODEL ANN

In [7]:

```
history_ann = ann.fit(X_train_img, y_train_cat,
        epochs=5,
        batch_size=32,
        validation_data=(X_test_img, y_test_cat),
        verbose=1)
acc_ann = ann.evaluate(X_test_img, y_test_cat,
        verbose=0)[1]
acc_ann
```

8. CNN MODEL & COMPILE

In[8]:

```
cnn = sequential([
    Conv2D(32, kernel_size=(3,3), activation="relu",
    input_shape=(28,28,1)),
    #First convolutim
    MaxPooling2D(pool_size=(2,2)), #Reduce dimension
    Conv2D(64, kernel_size=(3,3), activation="relu"),
    #2nd convolution
    MaxPooling2D(pool_size=(2,2)), #Reduce dimension
    Flatten(), #convert to vector, 3D to 1D
    Dense(128, activation="relu"),
    #Hidden layer
    Dropout(0.5), #Prevent overfitting
    Dense(10, activation="softmax")])
    #output layer
cnn.compile(optimizer="adam", loss="categorical_crossentropy",
    metrics=["accuracy"])
```

since in futher section in only visualization & comparison visualization we can easily generate with AI like google colab gemini with prompts

9. TRAINING HISTORY VISUALIZATION

#For

Prompt -

> Generate a function to plot training & validation accuracy & loss using Matplotlib
> #For Visualize Model learning Performance

10. ACCURACY COMPARISON VISUALIZATION

Prompt

> Generate a line chart comparing validation accuracy of Multiple models
> # Compare model accuracy over epochs

11. PREDICTION COMPARISON VISUALIZATION

prompt

> Generate a function to display sample image & compare predictions from multiple Models.

12. CONFUSION MATRIX VISUALIZATION

Prompt

> generate a confusion Matrix heatmap using seaborn for the trained model #Analyze classification performance

13. FINAL ACCURACY VISUALIZATION

prompt

> generate a bar chart comparing the final test accuracy of Multiple models with percentage labels

(8.5) TRAIN & EVAULATE CNN

[8.5]

```
history_cnn = cnn.fit(
        X_train_cnn,
        y_train_cat,
        epochs=5,
        batch_size=32,
        validation_data=(X_test_cnn, y_test_cat),
        verbose=1)

acc_cnn = cnn.evaluate(X_test_cnn, y_test_cat,
        verbose=0)[1]

acc_cnn
```

PART-3
RNN (RECURRENT NEURAL NETWORK)

→ RNN (recurrent neural network)

It is neural network architecture used to sequential data that remembers previous information through a hidden state, enabling it to understand context & sequence.

↳ Advantages

- Handle sequential data
- Remembers previous inputs
- Learns context & dependencies
- support variable-length sequences.
- Good for time-series & NLP Tasks.

↳ Applications of RNN

- sentiment analysis (classifies text as +ve, -ve or neutral)
- Machine Translation
- Text generation
- speech Recognition
- chatbots
- Handwriting Recognition

→ Sequential & Non-sequential Data

↳ **sequential Data** : Data in which the order of element doesn't matters & features are independent on previous one.

Ex- Age, salary, location

- changing their order usually doesn't change the meaning.

↳ **Non-Sequential Data**

Data in which the order of element doesn't matter & each element depends on previous ones.

Ex- | Hello , I am aniket |

correct sequence has a meaning if changed to

| I aniket Hello am | the meaning is lost.

- suitable for RNN

→ **Why use RNN instead of ANN**

(1.) Text input size May Vary

ANN requires a strictly fixed no. of inputs. text sentences have varying length

ex-
one sentence might be 3 words
another might have 4 words

We cannot push varying input sizes into a fixed ANN Architecture.

(2.) High computing Power

ML models require words to be vectorized (ex- ONE HOT ENCODING).

even for a small sentence, encoding words against the total vocabulary size create an erhormous no. of inputs. connecting all these to hidden layers exponentially increases the required weights, demanding massive computational power

(3) Prediction/Testing Problem

If we train a ANN on a sentence with maximum length of 7 words, but later test it on a 200 word, the ANN architecture will fail completely.

Because it cannot adapt its input nodes dynamically.

(4) Loss of segmantic Meaning (The Biggest Issue)

- segmantic Meaning - It is the meaning or context conveyed by words or a sentence

Ex - "I love RNN" → segmantic meaning
"AI love I" → segmantic meaning lost.

ANN treat every input word entirely independently & do not establish contextual connections between them.

Because the sequence is not retained, the segmantic meaning of the sentence is completely lost.

→ RNN Architecture

↳ Data Feeding

It is the process of providing input data to an RNN one element (word or time step) at a time in sequential order for processing.

Ex- Comments (Input)

Comment	sentiment
You are good	1 (Positive)
You are bad	0 (Negative)
you are not good	0 (Negative)

(1.) Vocabulary : The unique words in the dataset from the vocabulary.

Ex-

Word	one-hot encoding
You	[1, 0, 0, 0, 0]
Are	[0, 1, 0, 0, 0]
Good	[0, 0, 1, 0, 0]
Bad	[0, 0, 0, 1, 0]
Not	[0, 0, 0, 0, 1]

- Vocabulary Size = 5

*one hot encoding converts words into vectors.

(2) Time step : It represents one element (word) processed by ANN.

Ex- you are good

- Time step 1 → you
- Time step 2 → Are
- Time step 3 → Good

- Total Time step = 3

(3) Batch size : It is the no. of input sequences (sentences) processed together by the RNN in one iteration.

Ex-

you are good
you are bad
you are not good

since all 3 sentences processed together.

- Batch size = 3

(4) Input Features : Each word is represented using One-hot Encoding

Ex-

you are good
you are bad
you are not good

since the vocabulary contains 5 unique words

- Input Feature = 5

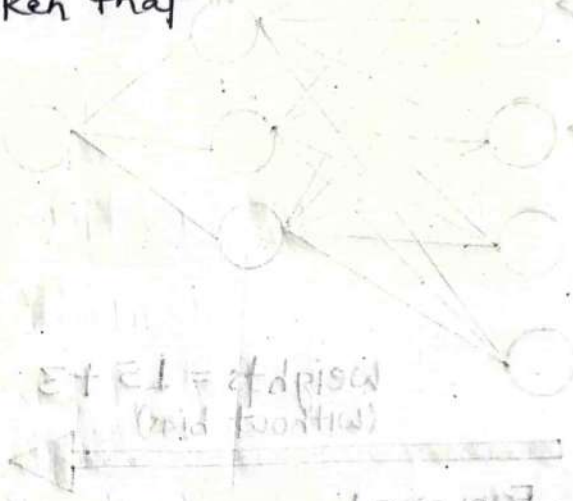
(5.) Input shape for RNN (keras)

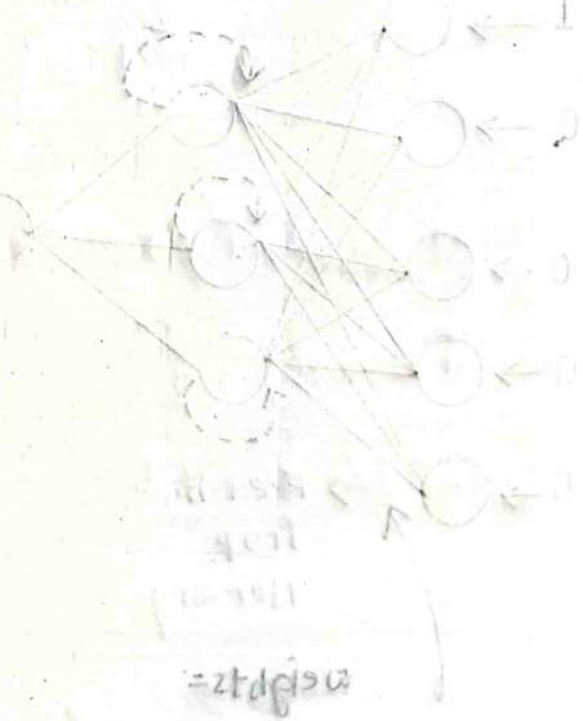
RNN input is represented as

keras ⇒ (batch size, timestep , Input features)

keras ⇒ (3 , 4, 6)

↳ timestep always takes higher. because 3rd sentence has 4 words we taken that

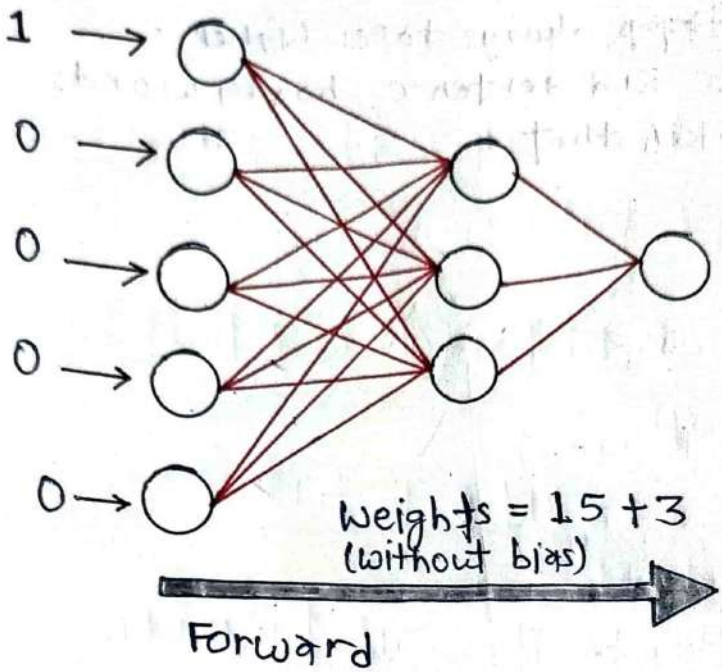




→ RNN Architecture (Working & weight calculations)

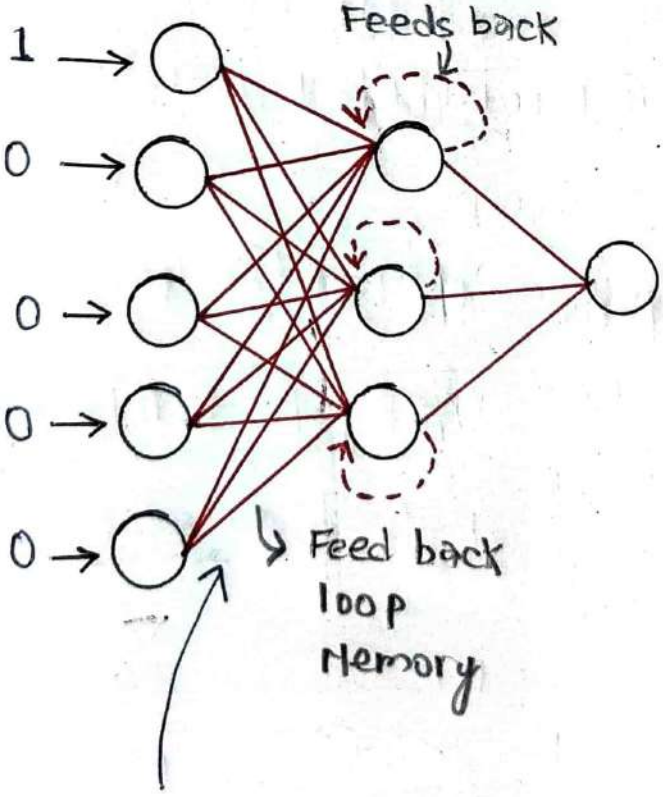
(1.) ANN vs RNN

↳ ANN (Feed-Forward Network)



- Data allows only in the forward direction
- No memory of previous inputs
- suitable for non-sequential data.
- example- Image classification, spam detection, Tabular data etc.

↳ RNN (Feedback / Recurrent Network)



- Data flows forward & feeds back to the hidden layer.
- Maintains memory using the previous state
- suitable for sequential data like text & speech.
- Example - Text generation, Language modeling, speech Recognition, Time series predic etc

- Its have an activation function inside their hidden layer.

weights = 15 + 3 + (3X3)

In every new time stamp it increases 3X3=9 mean 9.

(2.) Example: We use this example

Sentence = | you are good |

Time step	Word
T_1	you
T_2	Are
T_3	Good

- Total words = 3
- Total time step = 3

(3.) ANN Weight calculation

suppose

- Vocabulary size = 6
- Hidden neuron = 3
- output Neuron = 1

- **Input → Hidden**

6 X 3 = 18 weights

- **Hidden layer → output**

3 X 1 = 3 { Hidden = 3, Bias

- **Total parameters**

18 + 3 + 3 + 1 = 25 { output = 1, Bias

(4.) Weight sharing in RNN

Unlike ANN, RNN doesn't create new inputs weights for every word. Instead, it shares the same weights across every time step.

- Input weights = W_i

  used for every input word

- Hidden (Recurrent) Weights = W_h

  Transfer information from one hidden state to the next.

The sharing reduces the no. of parameter & allows the model to learn sequential dependencies.

(5.) Internal Working of RNN

An RNN processes a sentence one word at a time.

Each word is processed at a different time step.

At every time step, the network combines:

- current input
- Previous hidden state (memory)

This enables the RNN to understand the context of the sequence.

(6.) Time Step (T_i)

- Input : | you |

- one Hot encoder :

| [1, 0, 0, 0, 0] |

- calculation

$$O_1 = \tanh(X_1 W_i + b)$$

• Process

Input → Multiply with W_i → Add Bias → tanh() → output O_1

• O_1 becomes the memory for the next time step

(7) Time step (T_2)

• Input : [Are]

• One-Hot Encoder : [0,1,0,0,0]

Now the network receives
- current Input
- Previous output (O_1)

• calculations

$$O_2 = \tanh(X_2 W_i + W_h O_1 + b)$$

Network remembers

[You are]

instead of only : [Are]

• Process

current input + Previous Hidden state → Multiply with W_i & W_h → Add → tanh() → Output O_2

(8) Time step (T_3)

• Input : [Good]

The network combines
- Good
- Previous hidden state (O_2)

• Calculations

$$O_3 = \tanh(X_3 W_i + W_h O_2 + b)$$

Now the hidden state contains information about

you → are → Good

instead of only

Good

Thus, the semantic meaning of the sentence is preserved.

(9.) Hidden state Flow

you → O_1 → Are → O_2 → good → O_3

O = previous output

Each hidden state carries information from the previous state.

(10) Activation Function

The hidden layer uses "tanh" as default

• Purpose

- Introduces non-linearity
- Produces the hidden state
- Helps the network remember previous information.

• output Range

+1 to 1

(11.) Why RNN Understands context

- ANN processes words independently.

Ex - Good

ANN only sees the current word.

- RNN Remembers

Ex - you → are → good

Therefore, RNN preserves the semantic meaning of the sentence.

(12.) Text processing for RNN

Before feeding text into RNN, the text must be converted into numerical form.

- (a) Tokenization - It is the process of assigning a unique integer to every unique word in the vocabulary.

Ex

Word	Token
You	1
Are	2
Good	3
Bad	4
Not	5

- sentence

you are good → it becomes [1, 2, 3]

• **(b) Vocabulary size** : It is the maximum no. of words the model learns.

Ex- `vocab_size = 2000` only 2000 most frequent words are kept

• Unknown words are replaced with the OOV (out of vocabulary) token.

(c) Padding : Different sentence have different length

Ex - "you are good" (3 words) → "you are not good" (4 words)

Neutral networks require equal-lengths inputs.

Padding adds zeros to shorter sentences.

Ex (post padding) -

"you are good" → [1, 2, 3, 0, 0]

Max^m length = 5

PROJECT-1
TEXT PROCESSING USING RNN

↳ Overview

- created a small text dataset
- converted words into numbers.
- Used an Embedding layer to represent words.
- Applied a simple RNN to learn word sequences.
- classifies sentences as Positive or Negative.
- Visualized the RNN hidden states.

1. IMPORT LIBRARIES

In [1]:

```
import numpy as np
import tensorflow as tf
#Import tensorflow for Deep learning framework
from tensorflow.keras.preprocessing.text import
Tokenizer
# convert text to tokens
from tensorflow.keras.preprocessing.sequence import
pad_sequence
# Make all sequence same length
from tensorflow.keras.models import Model
# Build Keras Model
from tensorflow.keras.layers import Input, Embedding,
SimpleRNN, Dense
#Model layers
```

2. CREATE DATASET

In [2]:

```
sentence = [.....]    #Input text samples
labels = [1] * 15 + [0] * 15  # 1 = Positive , 0 = Negative
labels = np.array(labels) #convert to Numpy array
```

↳ Dataset Info:-

- Total sentences : 30
- Total words : 131
- Unique words (Vocabulary) : 86
- Positive samples : 15
- Negative samples : 15
- Average Sentence Length : ~4-5 words

3. TEXT PRE-PROCESSING

In [3]:

```
vocab_size = 2000  #Maxm vocabulary size
tok = Tokenizer(num_words = vocab_size, oov_token = "<oov>")
# create Tokenizer
tok.fit_on_texts (sentences)
  #Learn vocabulary
seqs = tok.texts_to_sequence(sentences)
#convert text to integer sequences
maxlen = max(len(s) for s in seqs)
#Find Longest sentence
X = pad_sequences(seqs, maxlen = maxlen, padding ='post')
# Pad shorter response
y = labels
#Target labels
```

4. VIEW SAMPLE INPUT & DEFINE MODEL PARAMETER

In[4]:

```
X[0]
#Display first tokenized sentence
embed_dim = 16
#embedding vector size
rnn_units = 8
#No. of RNN neurons
```

5. BUILD RNN MODEL

In[5]:

```
inp = Input(shape=(maxlen,),dtype="int32",name="input")
# Input layer
x= Embedding(input_dim=vocab_size,output_dim=embed_dim,
mask_zero=True,name="embed")(inp)
#Embedding Layer
rnn=simpleRNN(units=rnn_units,return_sequence=False,
return_state=False,name="simple_rnn")
#simple RNN layer
x_last = rnn(x)
# RNN output
out = Dense(1,activation="sigmoid",name="out")(x_last)
# output Layer
model = Model(inputs=inp,outputs=out)
#create Model
model.compile(optimizer="adam",loss="binary-crossentropy",
metrics=["accuracy"])  # configure training
model.summary()  #Show architecture
```

6. TRAIN MODEL

In [6]:

```
model.fit(X, y, epochs=25, batch_size=8, verbose=1)
```

Next Use this prompt

create an intermediate keras model to visualize the outputs of the embedding layer & the hidden states of the simple RNN layer from the existing model. Then use this intermediate model to predict on the input data & visualize the outputs

→ RNN Limitations

In today's world, RNN are rarely used for long sequential data because they suffer from several major limitations.

(1.) Weak Long-Term Memory

Standard RNN mainly maintains Short-Term Memory.

As sequence becomes longer, it gradually forgets information from earlier timestamps, making it difficult to connect distant words or events.

Ex- | Rahul --- (998 words later) --- lives in Delhi |

Q- Que- where does Rahul live?

The RNN may forget Rahul before reaching Delhi.

(2.) Vanishing Gradient Problem

During Backpropagation Propagation Through Time (BPTT), repeated multiplication of small gradients cause them to approach 0.

Results :

- Weights stop updating.
- cannot learn long-term dependencies.
- Training becomes ineffective.

(3.) Exploding Gradient Problem

Repeated Multiplication of large gradients cause them to grow exponentially.

Results :

- Unstable weight updates
- Loss increases rapidly
- Training may diverge

(4) Cannot capture Long-Range Dependencies

information fades over many timestamps, making it relate distant words or events.

Ex | Rahul - - - Delhi |

The relationship between Rahul & Delhi becomes weak.

(5.) Slow sequential Processing

RNN processes one timestamp at a time.

| Word1 → Word2 → Word3 → Word4 → - - - |

Results:

- cannot be fully parallelized
- slow training
- High inference time for long sequences.

→ LSTM (Long Short-Term Memory)

It is an advanced type of Recurrent Neural Network (RNN), introduced in 1997, that is designed to solve the-

- Weak long-term Memory
- Vanishing gradient
- long range dependencies problems of standard RNNs

It uses cell state, Hidden state & three gates (Forget, Input & output)

↳ Advantages

- solve the vanishing Gradient problem
- Learns long-term Dependies
- Better memory than standard RNN.
- suitable for long sequential data

↳ Disadvantages

- complex architectures (3 gates)
- More parameters.
- Higher computational cost.
- Slower training than GRU.

↳ LSTM maintains two different memory states

(A) Cell state (C_t) :- Long Term Memory

- stores information across long sequence
- Information flows almost unchanged through the network.

• Acts as a memory highway.

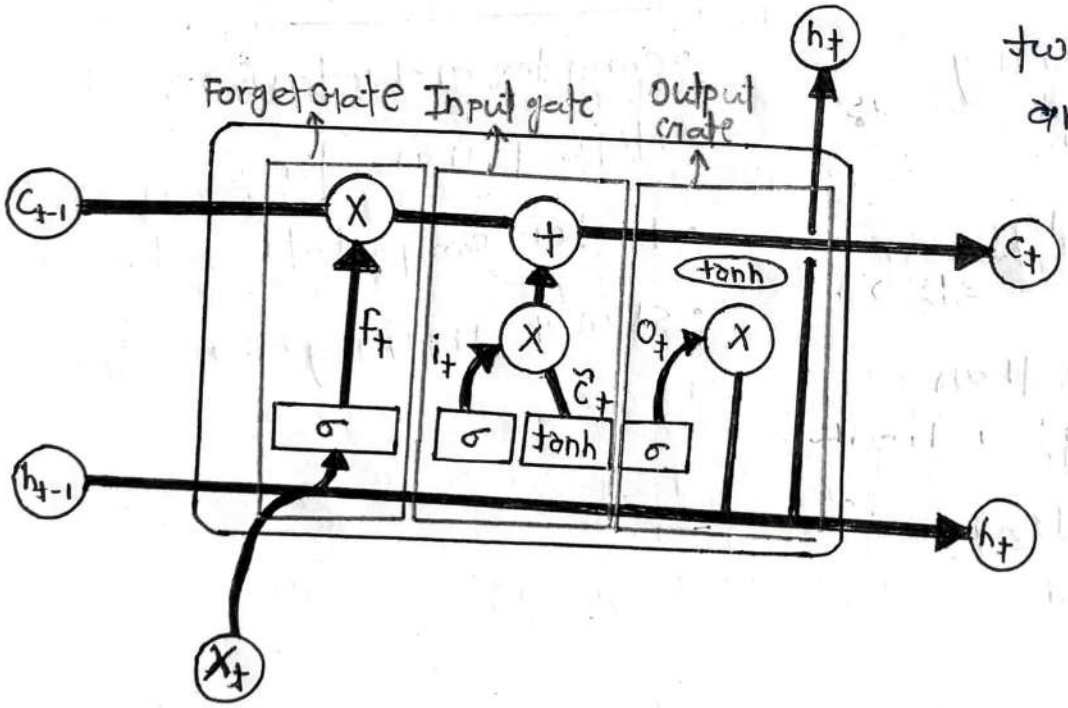
Cell state → Long Term Memory

(B:) Hidden state (H_t) : Short-Term Memory

- Stores information needed for the current time stamp.
- Passed to the next timestamp
- Used to make predictions.

Hidden State (H_t) → Short-Term Memory

→ LSTM Architecture : Its architecture consists of two memory states and three gates that control the flow of the information.



↳ components

- Input (X_t) = current input at timestamp t.
- Hidden state (H_t) = short-Term Memory passed to the next timestamp
- cell state (C_t) = Long term Memeory that stores important information over long sequences.

→ Three gates

(1.) Forget Gate (f_t) : Remove unnessary information from the cell state

(2) Input Gate (i_t) : Adds useful new information to the cell state

(3) Output Gate (O_t) : Generate the new hidden state (output) for the current timestamp.

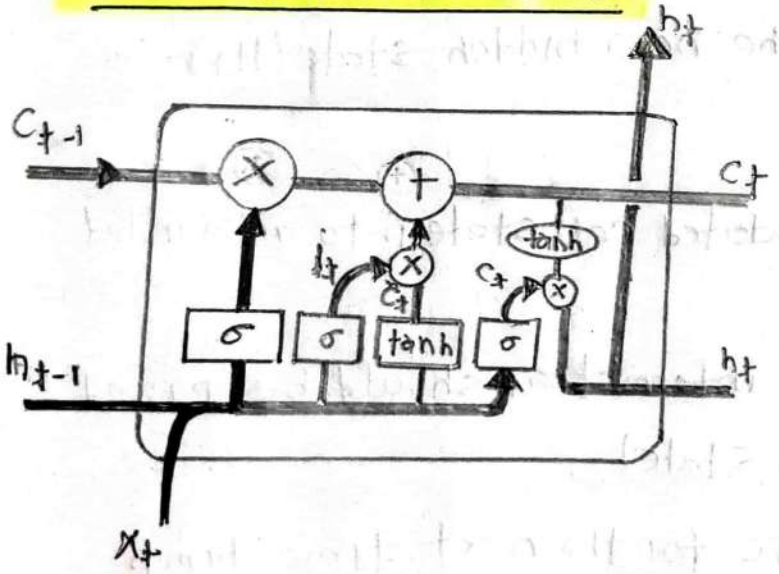
↳ components

- C_{t-1} = Previous cell state
- H_{t-1} = Previous Hidden state

→ LSTM gates

(1.) Forget Gate (f_t):

Removes unnecessary information from the cell state.



↳ working

- Takes current Input (X_t) and Previous Hidden state (H_{t-1})
- Uses sigmoid (because it outputs values between 0 & 1, deciding what to forget or keep)
- Output:
  - 0 → Forget
  - 1 → keep

↳ Formula

$$f_t = \sigma(W_f [H_{t-1}, X_t] + b_f)$$

↳ Ex : John moved from Delhi to London

The forget gate reduces the importance of Delhi & increase the importances of London.

(2). Input Gate : Adds useful new information to the cell state.

↳ Working

- Uses tanh (to generate new candidate information between -1 & 1)
- Uses sigmoid (to decide which candidate information should to stored)
- Updates the cell state with important old & new information.

↳ Formula

$$i_t = \sigma(W_i [H_{t-1}, X_t] + b_i)$$
$$\tilde{C} = \tanh(W_c [H_{t-1}, X_t] + b_c)$$
$$C_t = f_t C_{t-1} + i_t \tilde{C}_t$$

↳ Example

stores "new job" in the sentence "Rahul got a new job"

(3.) Output Gate : Generates the new hidden state (H_t).

↳ Working

- Uses tanh (to transform the updated cell state into a bounded range)
- Uses sigmoid (to decide what information should be passed as the hidden state)
- Produces the new hidden state for the next timestamp and output layer.

↳ Formula

$$O_t = \sigma(W_o [H_{t-1}, X_t] + b_o)$$
$$H_t = O_t \times \tanh(C_t)$$

↳ Example

Sentence: "Rahul lives in Delhi".

The output gate passes "Delhi" as the relevant information to predict the answer.

→ Problems with LSTM

(1.) Complex Architecture

It Uses 3 gates (Forget, Input, output) & Multiple Activation function.

(2.) Large number of Training Parameters

More gates mean more weights & biases to train

(3.) High Time complexity

More computations lead to slower training & inference.

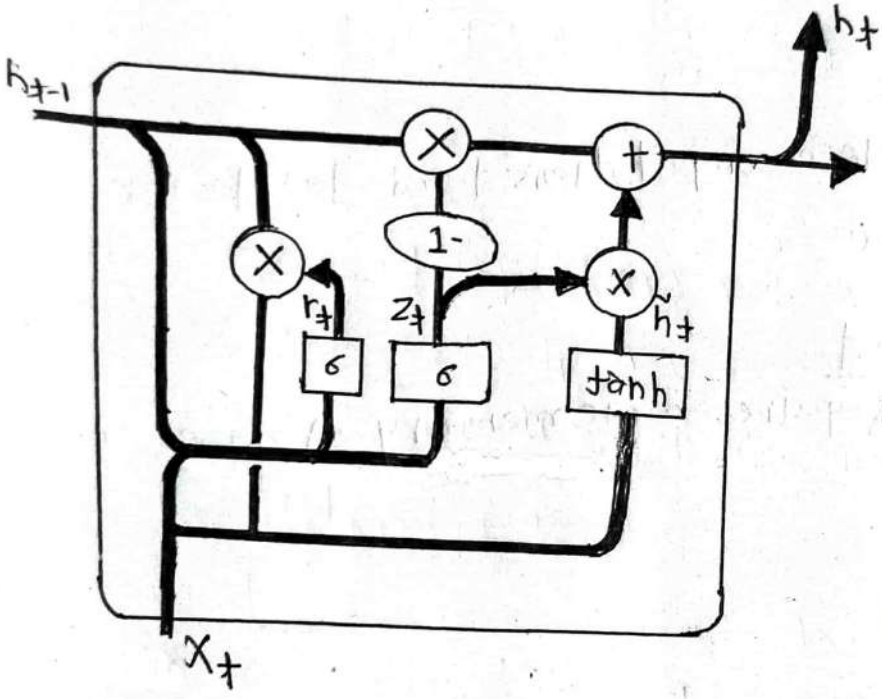
(4.) High computational cost

Requires more memory & processing power than GRU.

→ GRU (Gated Recurrent Unit)

It is a advanced type of RNN, introduced in 2014.

It simplifies LSTM by using one Hidden state & two gates (Reset & Update) It learns -term dependencies with fewer parameters, lower computational cost, & faster training than LSTM.



↳ Advantages

- simpler architecture than LSTM.
- only 2 gates
- Fewer parameter.
- Fasting training
- Lower computational cost.
- Performance comparable to LSTM.

↳ Disadvantages

- No seprate cell state
- slightly less effective than LSTM for very long sequence.
- Less flexible for complex memory tasks.

↳ GRU Terminologies

- σ (Sigma) = Sigmoid Activation function
- tanh = Hyperbolic Tanget Activation Function
- H_{t-1} = Previous Hidden state (Previous memory)
- X_t = current Input
- (X) = Element wise Multiplication
- (+) = Element wise Addition

- R_t = Reset gate
- z_t = Update gate
- $\tilde{H}_t$ = candidate Hidden state
- H_t = Current Hidden state (Updated Memory)

→ GRU Architecture

GRU consists of

↳ one memory state
- Hidden state (H_t)

↳ Two gates
- Reset Gate (R_t)
- Update Gate (z_t)

↳ GRU working

(1) Receive the current input (X_t).
(2) Reset gate filters unnessary past information.
(3) create the candidate Hidden state.
(4) Update Gate decides how much old memory to keep & how much new memory to add.
(5) Generate the new Hidden state (H_t).

→ GRU Gates

(1) Reset Gate (R_t) : controls how much previous memory should influence the current input.

↳ working
- Takes current Input (X_t) & Previous Hiddenstate (H_{t-1})
- Uses sigmoid (outputs values between 0 & 1)
- Filter old memory before combining it with the current input.

↳ Formula

$$R_t = \sigma(W_r[H_{t-1}, X_t] + b_r)$$

↳ Ex: "Rahul lives in Delhi, Today he moved to London."

The Reset gate reduces the influence of "Delhi" while processing London.

(2) Update Gate (Z_t)

controls how much old memory should be retained & how much new information should be added.

↳ working

- Takes current Input (X_t) & Previous Hidden state (H_{t-1})
- Uses sigmoid (outputs values between 0 & 1)
- Combines the functions of the forget gate & Input gate of LSTM

↳ Formula

$$Z_t = \sigma(W_z [H_{t-1}, X_t] + b_z)$$

↳ output

- High value → Keep old Memory
- Low value → Update with new information

↳ Ex: "Rahul lives in Delhi. Now he lives in London".

The update gate replaces the old Memory (Delhi) with new memory (London)

* Candidate Hidden State : Generate new candidate information.

↳ working

- uses tanh (outputs values between -1 & 1)
- combines the current input with the filter previous memory.

↳ Formula

$$\tilde{H}_t = \tanh(W_h [R_t * H_{t-1}, X_t] + b_h)$$

* Current Hidden state : The final memory is calculated the old memory & the candidate memory using the update gate.

↳ Formula

$$H_t = Z_t H_{t-1} + (1 - Z_t) \tilde{H}_t$$

PROJECT : 2
NEXT WORD PREDICTION USING RECURRENT NEURAL NETWORK (ANN & LSTM)

→ **Overview**

- Import & configure requires libraries.
- Prepare & preprocess text data.
- Convert text into numerical sequences.
- Build & train a Recurrent Neural Network (RNN)
- Evaluate Model performance.
- Generate predictions on new text.
- Analyze RNN hidden states & learned representations.
- Save the Trained Model for future use.

1. IMPORT LIBRARIES

In[1]:

```
import numpy as np
import pandas as pandas
import seaborn as sns
import matplotlib.pyplot as plt
```

2. LOAD DATASET

In[2]:

```
df = pd.read_csv ("quote-dataset.csv")
df.head()
df.shape
quotes = df ['quote']
```

3. TEXT PRE-PROCESSING

In [3]:

```
quotes = quotes.str.lower()
#convert all text into lowercase
import string
#Import punctuation library
translator = str.maketrans('','',string.punctuation)
#create a translation table to remove punctuation
quotes = quotes.apply(lambda x: x.translate(translator))
#Remove punctuation from every quote
quotes.head()
#Displayed cleaned quotes
```

4. TOKENIZATION

In [4]:

```
from tensorflow.keras.preprocessing.text import Tokenizer
#Import Tokenizer
vocab_size = 10000
#Maxm vocabulary size
tokenizer = Tokenizer(num_words = vocab_size)
#create Tokenizer
tokenizer.fit_on_texts(quotes)
# Build vocabulary from the quotes
word_index = tokenizer.word_index
#Store the word-index mapping
print(len(word_index))
#Display vocabulary size
list(word_index.items())[:10]
#Display first 10 vocabulary entries
```

5. CONVERT TEXT INTO INTEGER SEQUENCES

In [5]:

```
sequence = tokenizer.texts_to_sequences(quotes)
#Convert every quotes into integer tokens
for i in range(3):
    print(quotes[i])
#Display first three cleaned quotes
for i in range(3):
    print(sequence[i])
#Display tokenized representation.
```

6. CREATE TRAINING SEQUENCE

In [6]:

```
X = []  #store input sequences
y = []  #store target words
for seq in sequence:
#Generate training samples
        for i in range(1, len(seq)):
        # create multiple input-target pairs
            input_seq = seq[:i]
            #words before current position
            output_seq = seq[i]
            #Next word
            X.append(input_seq)
            #store input
            y.append(output_seq)
            #store target
len(X) #Display total training time
len(y) #Display total labels
```

7. FIND MAXIMUM SEQUENCE LENGTH

In[7]:

```
max_len = max(len(x) for x in X)
#Determine the longest input sequence
print(max_len)
#Display maxm sequence length
```

8. PADDING INPUT SEQUENCE

In[8]:

```
from tensorflow.keras.preprocessing.sequence import
pad_sequence
#Import the padding
X_padded = pad_sequence(X, maxlen=max_len, padding='pre')
#Pad all input sequence with zeros at the beginning, so
  every sequence has the same length
y = np.array(y)
#Convert target labels into Numpy array.
X_padded.shape
# Display input shape
```

9. ONE-HOT ENCODE TARGET LABELS

In[9]:

```
from tensorflow.keras.utils import to_categorical
#Import one hot encoding
y_one_hot = to_categorical(y, num_classes=vocab_size)
#convert target word indices into one-hot vectors.
y.shape #Display target shape
y_one_hot.shape
```

10. IMPORT DEEP LEARNING MODELS & HYPERPARAMETERS

In [10]:

```
from tensorflow.keras.models import Sequential
#Import sequential model
from tensorflow.keras.layers import (Embedding,
 SimpleRNN, LSTM, Dense)
 #Import required neural Network layers
 embedding_dim = 50
 # Dimension of word embeddings
 rnn_units = 128
 #No. of neurons in recurrent layers
```

11. SIMPLE ANN MODEL

In [11]:

```
rnn_model = Sequential()
# create the RNN model
rnn_model.add (Embedding (input_dim = vocab_size,
                          output_dim = embedding_dim,
                          input_length = max_len))
# convert word ID's into dense vectors
rnn_model.add (SimpleRNN (units = rnn_units))
# Add simple RNN layer
rnn_model.add(Dense(units=vocab_size, activation='softmax'))
#Output layer predicts the next word
rnn_model.compile (optimizer = 'adam', loss = 'categorical_
 crossentropy', metrics = ['accuracy'])
#Compile & configure optimizer, loss function & metrics
 rnn_model.summary()
 #Display model architecture
```

12. LSTM MODEL

In [12]:

```
lstm_model = Sec
lstm_model.add
output_dim = er
# word embeddi
lstm_model.add
# LSTM layer for
lstm_model.add

#Output layer predicts the next word
lstm_model.compile(optimizer='adam',
                   loss='categorical_crossentropy',
                   metrics=['accuracy'])
lstm_model.summary()
# Display Model summary
```

12.5 RNN TRAINING

In [12.5]

```
epochs = 10
batch_size = 128
history_rnn = rnn_model.fit(X_padded,
    y_one_hot, epochs = epochs,
    batch_size = batch_size, validation_
    split = 0.1)
```

13. LOAD THE TRAINED MODEL & CREATE INDEX-TO-WORD MAPPING

In [13]:

```
from tensorflow.keras.models import load_model
# Import Utility to load saved keras model.
lstm_model = load_model("lstm_model.h5")
# Load the trained LSTM model
lstm_model.save("lstm_model.h5")
#save the model again
index_to_word = {}
# Create an empty dictionary
for word, index in word_index.items():
#Reverse the tokenizer dictionary
        index_to_word[index] = word
    #store index as key & word as value
```

14. IMPORT PADDING & NEXT WORD PREDICTION FUNCTION

In [14]:

```
from tensorflow.keras.preprocessing.sequence import
 pad_sequence
#Import padding
def predictor(model, tokenizer, text, max_len):
#Predict the next most portable word.
        text = text.lower()
        #convert input text to lowercase
        seq = tokenizer.texts_to_sequence ([text])[0]
        #convert text into integer sequence
        seq = pad-resources ([seq], maxlen =max_len,
                             padding = 'pre')
        #Pad the sequence to the required length
        pred = model.predict(seq, verbose =0)
        #Predict probabilities for all vocabulary words
        pred_index = np.argmax(pred)
        #select the word with the highest probability
        return index_to_word [pred_index]
        #Convert index_to_word [pred_index]
```

15. TEST NEXT WORD PREDICTION

In [15]:

```
Seed_text = "what are you"
#sample input sentence
next_word = predictor(lstm_model, tokenizer, seed_text,
            max_len)
# Predict the next word
print (next_word)
#Display Prediction
```

16. TEXT GENERATION FUNCTION & GENERATE TEXT

In [16]:

```
def generate_text(model, tokenizer, seed_text,
                  max_len, n_words):
# Generate multiple words from a seed sentence
    for _ in range(n_words):
    # Generate one word at a time
        next_word = predictor(model, tokenizer, seed_text,
                              maxlen)
        # Predict the next word
        if next_word == "  ":
            break
        # stop if no valid word is predicted
        seed_text += " " + next_word
        # Append the predicted word
    return seed_text
    # Return the generated sentence.

seed= "are you a "
#starting sentence
generated_text = generated_text(lstm_model, tokenizer,
                  seed, max_len, 10)
# generated 10 additional words
print(generated_text)
# Display generated text
```

17. SAVE TOKENIZER

In[9]

```
import pickle #Import pickle for saving Python objects.
with open("tokenizer.pkl", "wb") as f:
    pickle.dump(tokenizer, f)
# save the trained tokenizer
```

18. SAVING MAXIMUM SEQUENCE LENGTH

In[18]:

```
with open(max_len.pkl", "wb") as f :
    pickle.dump(max_len, f)
#save maximum sequence length
```

↳ Next: We will deploy this Model on streamlit & make app.py using AI.

DEEP LEARNING

PART - 4
TRANSFORMERS & ATTENTION MECHANISM

→ Types of Neural Network Architecture

(1.) One-to-One : A single input produces a single output

Ex- Image → cat

↳ Used in -
- Image classification

(2.) One-to-Many : A single input generates a sequence of Multiple outputs.

Ex- Image → "A dog is playing in the park"

↳ Used in
- Image captioning

(3.) Many-to-one A sequence of inputs is processed to produce a single output.

Ex- Review "A dog is playing in the park" → Positive

↳ Used
- sentiment Analysis

(4) Many-to-Many (sequence-to-sequence)

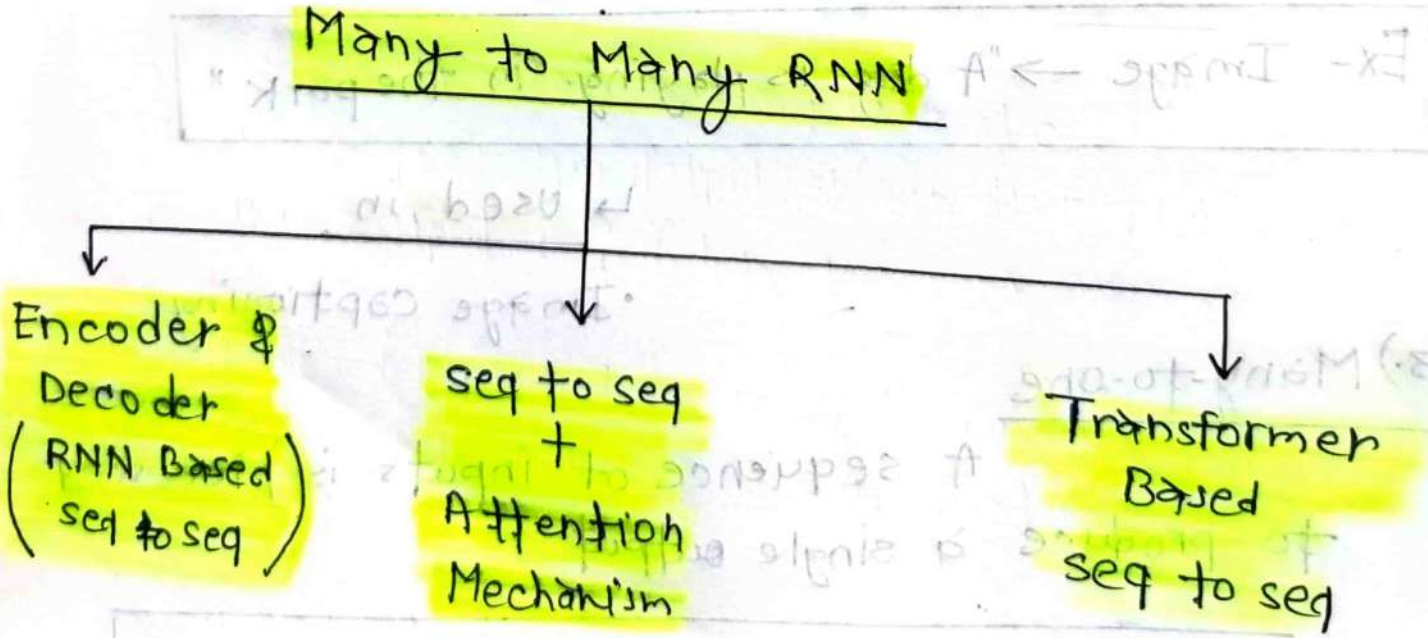
A sequence of inputs is converted into another sequence of outputs, when both the input & output can have different lengths

Ex- English sentence ("A dog is playing in the park") → French sentence ("Un chien joue dans le parc")

↳ **Used**

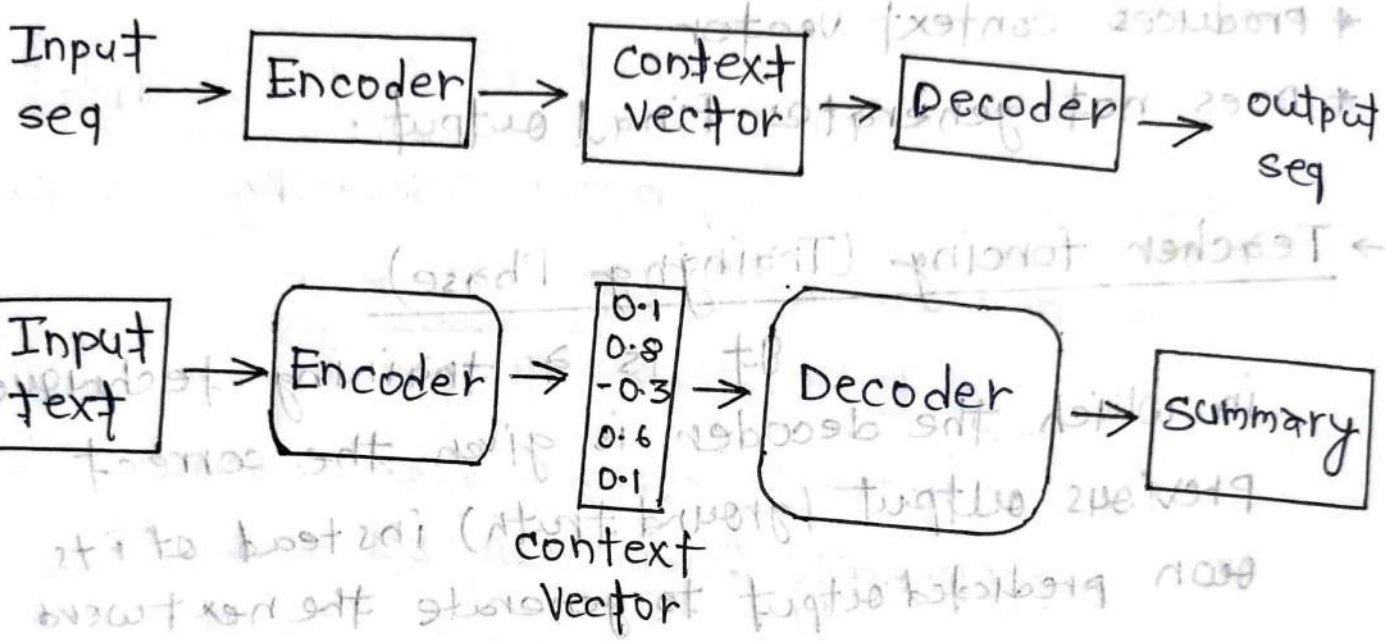
- Language Translation

Note:- Encoders-Decoders belongs to this architecture because it converts one input sequence into another output sequence.



→ Encoder & Decoder

These Models first understand the input sequence & then generates a new output sequence based on that understanding.



↳ **Encoder** : converts an Input sequence into a Meaningful Representation.

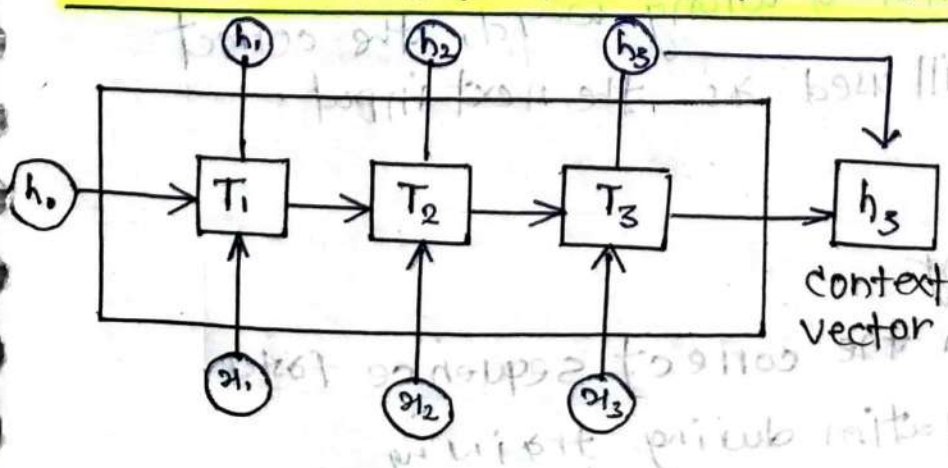
Ex-

Input → output

I	Love	Technology
x_1	x_2	x_3

मुझे	टैक्नोलॉजी	पसंद	है	<end>
y_1	y_2	y_3	y_4	

• Encoder Block (simple RNN, LSTM, GRU)



- x = Input
- T = Time step
- h = hidden state it stores information learned up to that point in the sequence

• Characteristics of Encoder

* Reads input sequentially
* Uses RNN, LSTM or GRU (classical seq to seq)
* Understand the complete sentence.
* Produces hidden states.
* Produces context vector.
* Does not generates final output.

↳ Teacher forcing (Training Phase)

It is a training technique in which the decoder is given the correct previous output (ground truth) instead of its own predicted output to generate the next word

Ex- Input: "I love Technology" → Target output: "मैं टेक्नोलॉजी पसंद है"

If Decoder gives "मैं" or anything else
It also do prediction output at own then pick own
In this output is "मैं" this go over & over

* even if decoder predict a wrong word, the correct previous word is still used as the next input.

• Uses

* speeds up model training
* Helps the model learn the correct sequence faster.
* Reduces error propagation during training
* Improves convergence & stability

• Used in -

* RNN based seq to seq
* LSTM Encoder-Decoder
* GRU Encoder-Decoder
* Transformer training

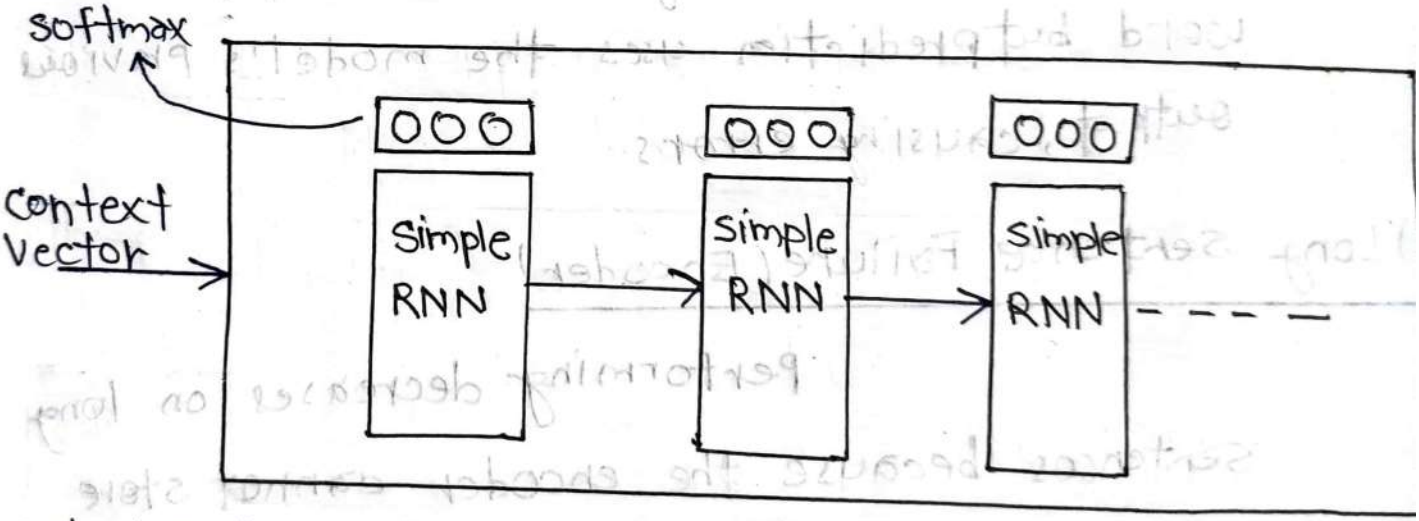
• Examples

* Language Translation
* Text summarization
* Question Answering
* chatbots
* speech Recognition

↳ **Decoder**

Generate the output seq one element at a time decoder is different in training & prediction.

- Decoder doesn't know output.
- It uses its own previous prediction.



• **Decoder characteristics**

* Receives context vector.
* Start from START token.
* Generates one word at a time
* Uses softmax
* Stops at End token

→ Problems in Encoder-Decoder

(1.) Information bottleneck (Encoder)

Entire input is compressed into one fixed size context vector, causing information loss.

(2.) No word-to-word Alignment (Encoder-Decoder)

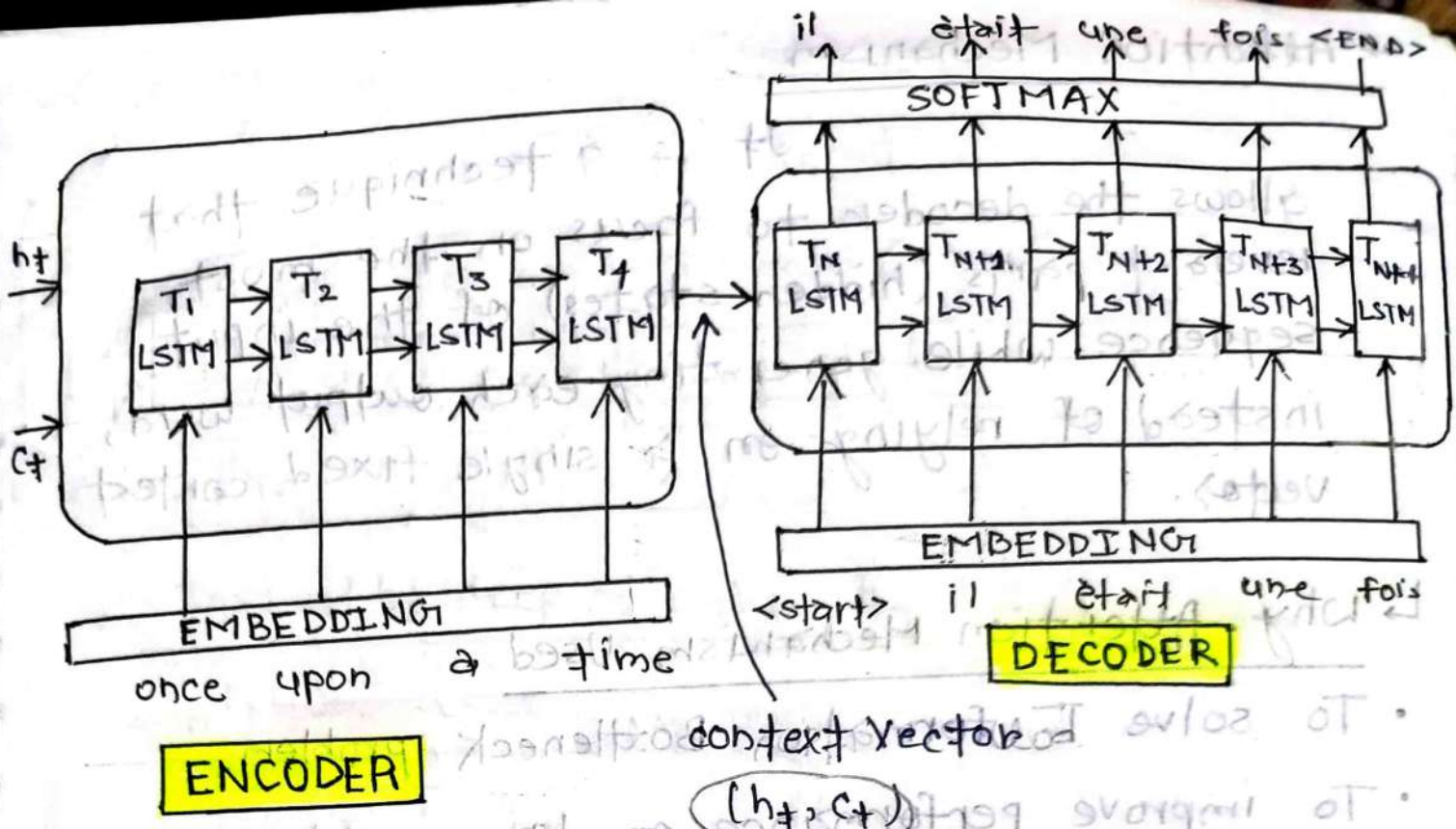
Decoder cannot identify which word to focus on while generating each output word.

(3) Exposure Bias (Decoder)

Training uses correct previous word but prediction uses the model's previous output, causing errors.

(4) Long Sentence Failure (Encoder)

Performing decreases on long sentences because the encoder cannot store all information in one context vector.


il
était
une
fois
<END>
SOFTMAX
h_t
C_t
T_1 LSTM
T_2 LSTM
T_3 LSTM
T_4 LSTM
T_N LSTM
T_{N+1} LSTM
T_{N+2} LSTM
T_{N+3} LSTM
T_{N+4} LSTM
EMBEDDING
EMBEDDING
once upon a time
<start> il était une fois
ENCODER
DECODER
context vector
(h_t, C_t)
↳ summary

→ Attention Mechanism

It is a technique that allows the decoder to focus on the most relevant parts (hidden states) of the input sequence while generating each output word, instead of relying on a single fixed context vector.

↳ Why Attention Mechanism Used

- To solve Information Bottleneck problem
- To improve performance on long sentences
- To provide word-to-word alignment between input & output.
- To generate a dynamic context vector for each output word.
- To improve translation & sequence generation accuracy.

↳ cons of Attention Mechanism

- still depends on RNN/LSTM/GRU (in the original Attention-based seq to seq models)
- sequential Processing makes training slow
- cannot be fully parallized
- computational cost increases for longer sequences.
- More complex than the basic Encoder-Decoder model.

↳ Working of Attention Mechanism

- **Step: 1** - Encoder Processes Input

Example - I love Machine Learning

* Encoder generates hidden states

h_1, h_2, h_3, h_4

* Instead of keeping only h_4

* Attention Mechanism keeps:

h_1, h_2, h_3, h_4

- **step: 2** - Decoder Predict First word

* when predicting the first output word, the decoder look at

h_1, h_2, h_3, h_4

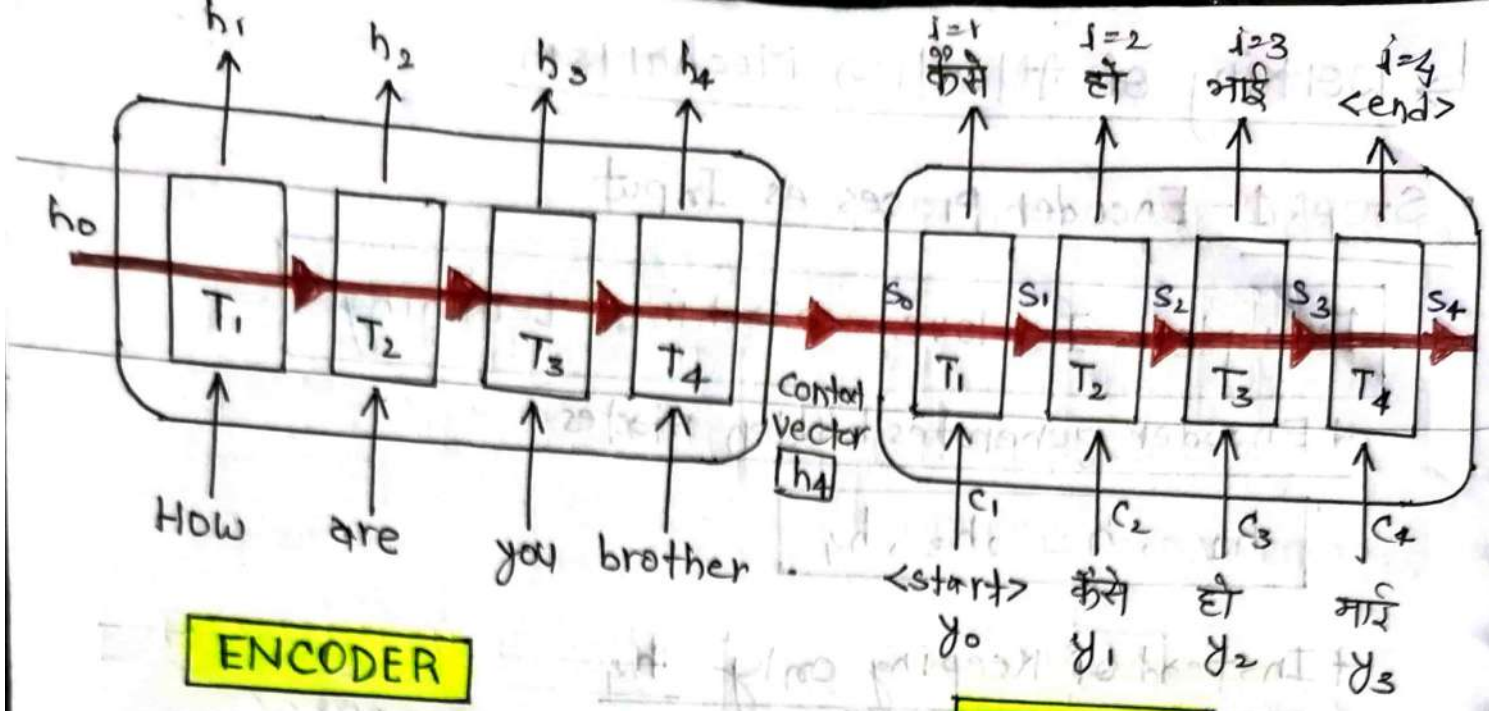
* instead of only h_4

- **step: 3** - calculate Attention score (α)

* Each hidden states receives a weight

Ex - $h_1 \rightarrow 0.10$
$h_2 \rightarrow 0.20$
$h_3 \rightarrow 0.60$
$h_4 \rightarrow 0.10$

* Higher weight ⟶ More Attention



↳ **Attention score (α)** : It is a weight that tells the Model how important each encoder hidden state is for predicting the current output word.

Higher α	→	More Focus
Lower α	→	Less Focus

↳ **context vector**

It is a fixed-size vector that summaries the entire input sequence & is passed from the encoder to the decoder.

↳ Dynamic context Vector

It is a context vector generated for each output word using the Attention Mechanism

- **Formula**

$$C_i = \Sigma (\alpha_i h_i) \quad \text{or} \quad C = \alpha_1 h_1 + \alpha_2 h_2 + \ldots + \alpha_n h_n$$

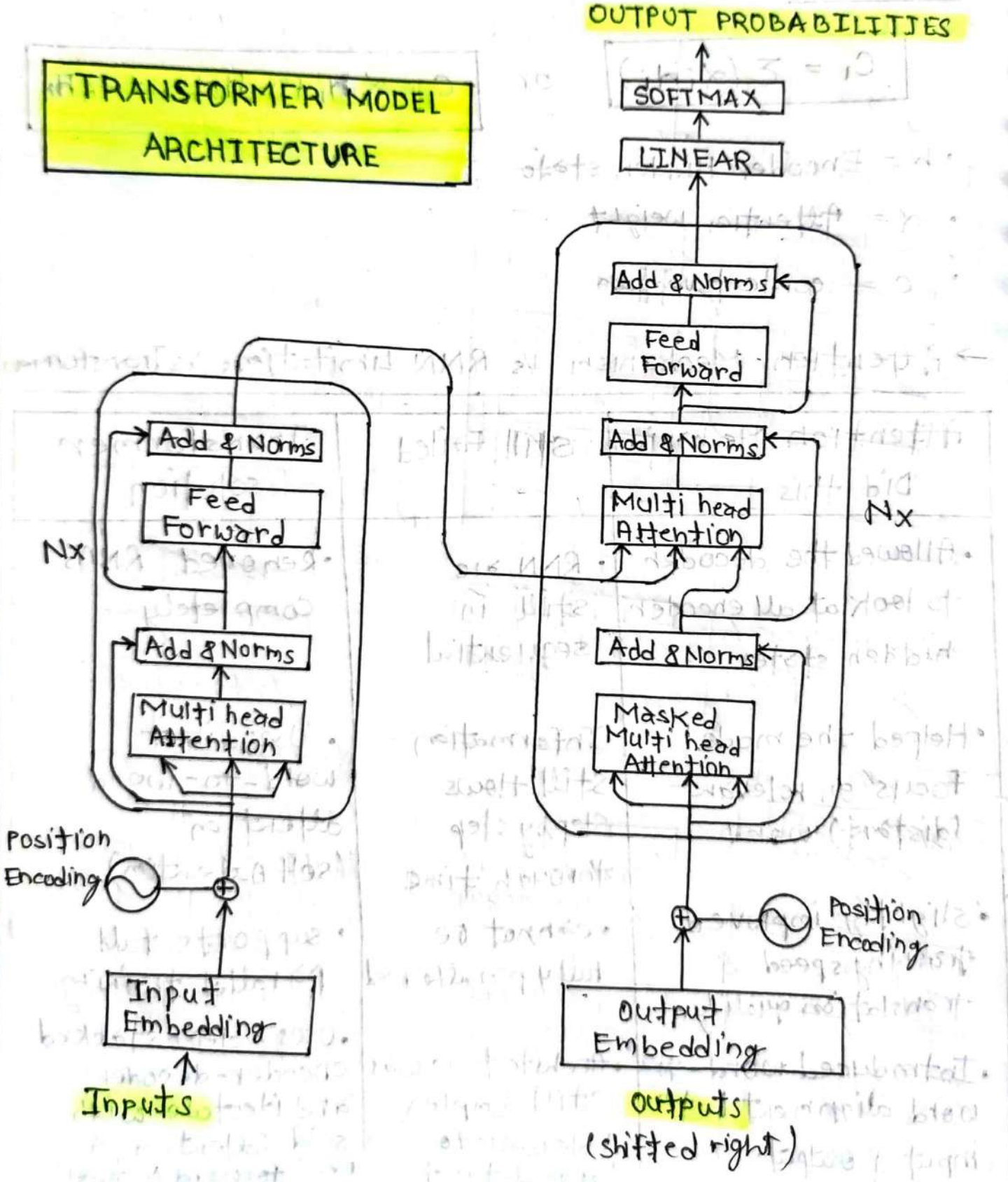
- h = Encoder Hidden state
- α = Attention weight
- c = context vector

→ Attention Mechanism Vs RNN Limitations Vs Transformer

Attention Mechanism Did this	Still Failed	Transformer solution
• Allowed the decoder to look at all encoder hidden states	• RNN are still in sequential	• Removed RNN's Completely
• Helped the model focus on relevant (distant) words	• Information still flows step by step through time	• Uses direct word-to-word attention (self Attention)
• slightly improved training speed & translation quality.	• cannot be fully parallelized	• supports full Parallel training
• Introduced word-to-word alignment betn input & output.	• Architecture was still complex & slower for large dataset	• Uses a clean stacked encoder-decoder architecture with self Attention & Feed forward Networks

→ Transformers

It is deep learning architecture that uses self-Attention instead of RNN's to process sequences in parallel, making it faster & more efficient for NLP tasks.



↳ Why Transformers Introduced ?

They were introduced to overcome the limitations of RNN-based Encode-Decoder Models

Reasons

- Remove RNN dependency
- Enable parallel preprocessing
- Improve training speed
- Handle long-range dependencies
- Train on massive datasets
- Build more accurate language models.

↳ cons of Transformers

- Requires large dataset for training.
- High computational cost.
- Needs powerful GPUs / TPUs
- self-Attention has quadratic complexity for long sequence
- Large memory consumption.

↳ Examples

- GPT
- BERT
- Gemini
- claude
- Deepseek
- Keemi K2
- LLaMa
- Vision Transformer (ViT)

↳ Uses

- chatbots
- Language Translation
- Text summarization
- Question Answering
- Text generation
- code generation
- Image classification etc.

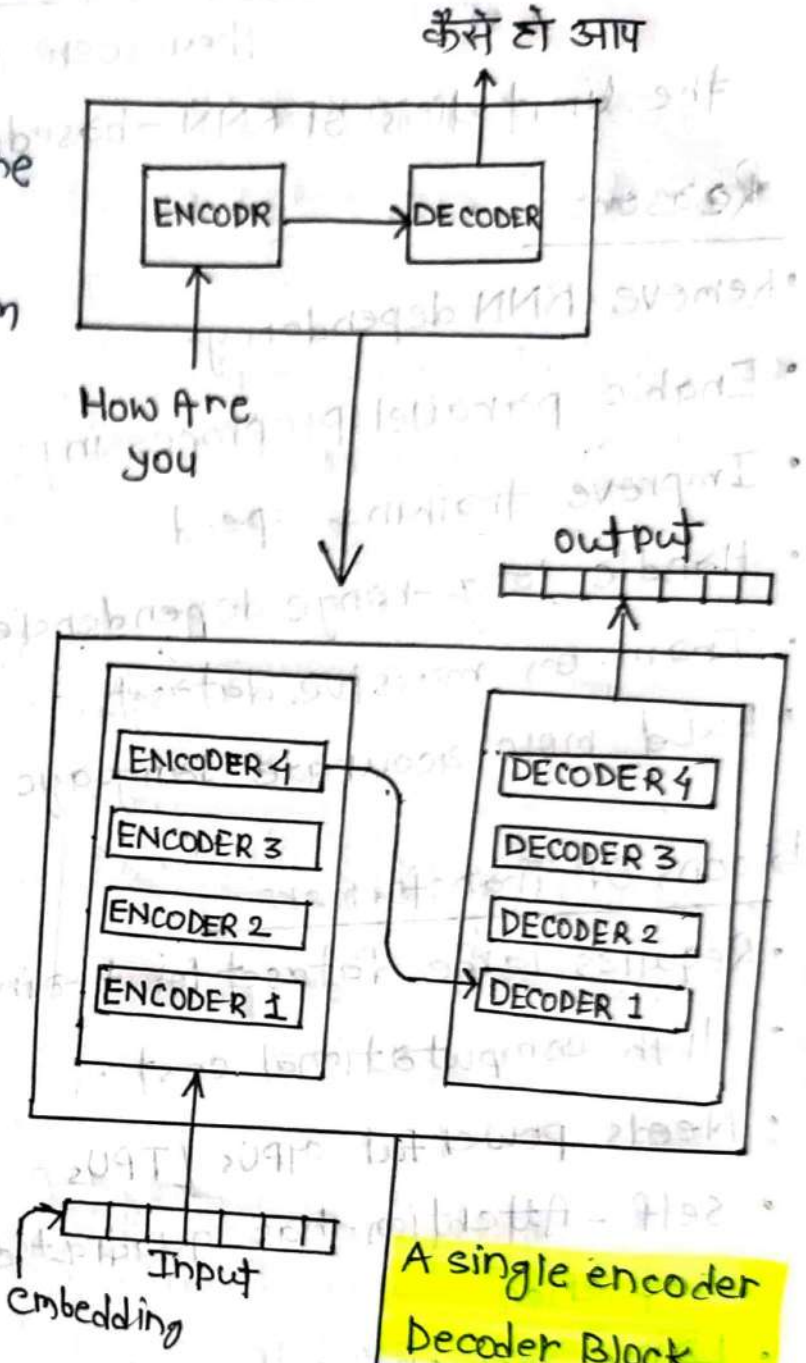
→ Encoder -Decoder Block

↳ Encoder Block

It processes the input sequence using Multi-Head self Attention and a Feed Forward Neural Network (FFN) to generate contextual representations.

- **Uses for**

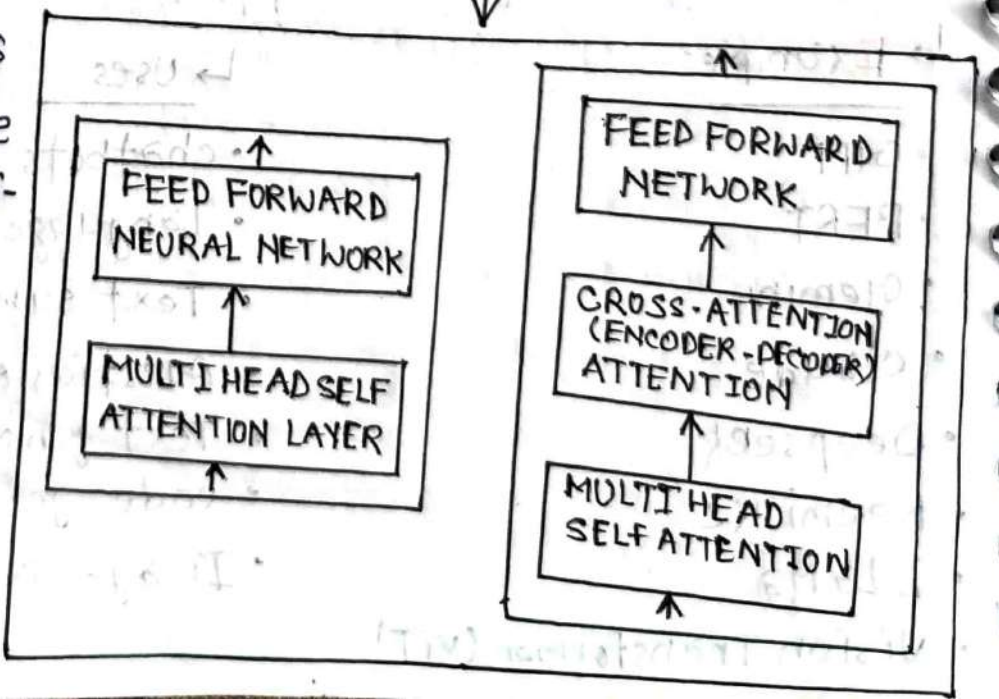
* Understands input context.
* Learns relationships between words
* Produces contextual representations for the decoder.



A single encoder Decoder Block

↳ Decoder Block

It generates the output sequence using Masked Multi-Head self Attention, cross-Attention, and a feed Forward Neural Network (FFN)



- Uses for
  - * Generates output tokens one by one
  - * Focuses on encoder outputs using - Attention
  - * predicts the next word in the sequence.

↳ stacked Encoder-Decoder Architecture

Instead of one encoder & one decoder, Transformers use multiple stacked blocks

- original transformers Vs Modern Transformer Models (2019)

6 Encoder + 6 Decoder Layer	24, 48, 96 or 128 + layers

These stacked blocks helps to Improve the model's understanding of language

- * More layers → Better understanding

↳ Layer wise Learning

- Lower layers : capturing basic Relationship
  - * Learns words
  - * Learn grammer
  - * Learn nearby relationships.
- Middle Layers : Understanding the context
  - * Learn sentence context.
  - * Understand phrases.
  - * capture broader meaning.

- **Higher layers** : capture the abstract meaning
  - * Learn semantic meaning.
  - * capture abstract concepts
  - * Understand long-range dependencies.

↳ **Position Encoding**

It adds position information to word embeddings so the Transformer can understand the order of words.

It Transformers process all words simultaneously without positional information.

It acts like

Ex -

Dog bites man	=	Man bites dog

↳ **Single Encoder cell**

It consists of

- Multi-Head self-Attention
- Feed Forward Neural Network (FFN)

It processes the input & generates contextual representations.

→ Multi-Head Self Attention

It is a mechanism that allows every word in a sentence to attend to every other word form multiple perspectives simultaneously, helping the model understand context, grammer & semantic relationships before making a prediction.

↳ **Example:** older models like RNN struggled with long sentences

sentence: "The food at the restaurent was terrible.. but the dessert was amazing".

- The word "amazing" & "desert" are far apart, RNNs may lose this relationship while processing the sentence sequentially.

↳ How self Attention solves this

self attention allows-

- Every word to directly attend to every other word.
- The word "amazing" to focus on "dessert" even though they are far apart
- Distance between words become largely irrelvant.
- The Model captures long-range dependencies more efficient.

↳ Need of Multi-Head Attention

Instead of using Head only one attention Mechanism, the Transformer uses Multiple attention Heads. Each head learns a different type of relationship, such as.

- Head 1: Grammer
- Head 2: Meaning
- Head 3: subject-object relationships
- Head 4: Long-range dependencies
- Head 5: other semantic & structural patterns

The outputs of all heads are combined to produce a richer contextual representation of sentences

↳ Query (Q): (search signal)

It represents what a word is looking for in the rest of the sentence. It is used to find the most relevant words for understanding the current word.

Ex. "The cat chased the mouse" Query: "who performed the action, & who recieved it".

↳ Key (K) - Information Label

It describes what information a word contains. It is compared with Queries to determine how relevant that word is.

Key:
- cat → "I am a the subject".
- mouse → "I am the object".
- chased → "I am the action"

↳ Value (V) - Actual Information

It contains the actual information that is passed to the output after the query-key matching determines the importance of each word.

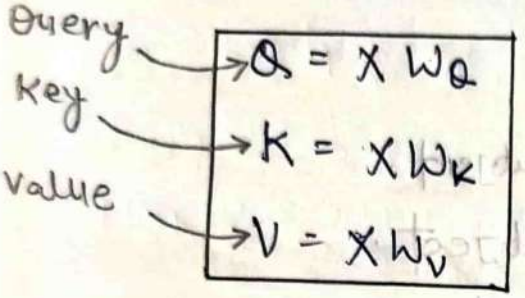
: **Value**

- cat → Information about subject.
- mouse → ,, ,, object.
- chased → ,, ,, action.

→ Working of Multihead self Attention

↳ Step: 1 - Generate Query (Q), Key (K) & value (V) vectors

- Here each input embedding is multiplied by three learnable metrics



- example

Please study Man

Please → [1 | 0]

study → [0 | 2]

Man → [1 | 1]

- calculations

Query = W_q

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Key = W_K

$$\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Value = W_v

$$\begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

- calculating Queries = ("please") vector $\times W_q$

This calculation also do for study & Man

$$[1, 0] \times \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$= [1\times1 + 0\times0, 1\times0, 0\times1] = [1, 0]$$

Please = [1, 0]

study = [0, 2]

Man = [1, 1]

- **calculate keys** : ("Please") vector X W_K
  ↳ for also study & Man

Please = [1, 1]

study = [0, 2]

Man = [1, 2]

- **calculate values** : ("Please") vector X W_V

Please = [1, 0]

study = [2, 2]

Man = [2, 1]

↳ Step: 2 - compute / calculate Raw Attention score

- Compare each Query with all keys using the dot product
- Higher scores indicate stronger relationships between words.

Attention score = $Q \cdot K^T$

- Q = Query Matrix
- K = key Matrix
- K^T = Transpose of the key Matrix (flips the key matrix so dot products with Query can be computed)

- **("Please") Attention score**

Please = Q = [1, 0]

Keys

Please = [1, 1]

study = [0, 2]

Man = [1, 2]

[1, 0] · [1, 1] = 1

[1, 0] · [0, 2] = 0

[1, 0] · [1, 2] = 1

• ("study") Attention score

$[0,2] \cdot [1,1] = 2$

$[0,2] \cdot [0,2] = 4$

$[0,2] \cdot [1,2] = 4$

• ("Man") Attention score

$[1,1] \cdot [1,1] = 2$

$[1,1] \cdot [0,2] = 2$

$[1,1] \cdot [1,2] = 3$

Please = 1, 0, 1

Study = 2, 4, 4

Man = 2, 2, 3

↳ **Step : 3 - Applying softmax**

- convert the raw attention scores into attention weights (probabilities) using the softmax function.
- The weights determine how much attention each word should recieve.

$$\text{Attention Weights} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

- $\sqrt{d_k}$ = scaling factor (prevents the attention scores from becoming too large before softmax, d_k = dimension of key vectors)

• "Please" Attention weights

$[1, 0, 1] \rightarrow [0.42, 0.16, 0.42]$

• "study" Attention weight

[2, 4, 4] → [0.06 , 0.47 , 0.47]

• study gives → 6% Attention to Please

47% Attention to itself

47% Attention to Man

↳ Step: 4- creating the context vector

- Multiply the attention weights with the value (V) vectors.
- The weighted sum produces a context vectors (contextual Embedding) for each word

Context Vector = Attention weights × V

Values → Please = [1, 0]

Study = [2, 2]

Man = [2, 1]

0.6 × [1, 0] = [0.06, 0]

0.47 × [2, 2] = [0.94, 0.94]

0.47 × [2, 1] = [0.94, 0.47] +

[1.94, 1.41] → contextual vector for study

↳ Step: 5 - Multi Head Processing

- The self attention process (Q, K, V → Attention) is performed simultaneously by multiple attention heads.
- Each head learns a different relationship in the sentence such as grammer, meaning, or long range dependencies.

Example: (8 heads): 512 - dimensional embedding

↓

8 heads X 64 dimension

↳ Step: 6 - Concatenate the heads

- The outputs of all attention heads are concatenated (joined together)
- This combine the knowledge learned by each head into a single vector.

Example: [64] + [64] + [64] + ... + [64]

↓

Concatenated Vector (512)

↳ Step :7- Linear projection

- The concatenated vector is passed through a linear layer (output projection).
- Formula:

$$\text{Output} = \text{Concat}(\text{Heads}) \times W_o$$

- Purpose = Mix information from all heads & produce the final attention output.

↳ Step :8- Add & Layer Normalization (Add & Norm)

- The attention output undergoes two operations:

• (1) Residual connection (Add)

$$Z = \text{LayerNorm}(\text{Output} + \text{original Input})$$

• Purpose

* Preserves original information.
* Helps prevent the vanishing gradient problem

• (2) Layer Normalization (Norm)

Normalize the output values.

• Purpose

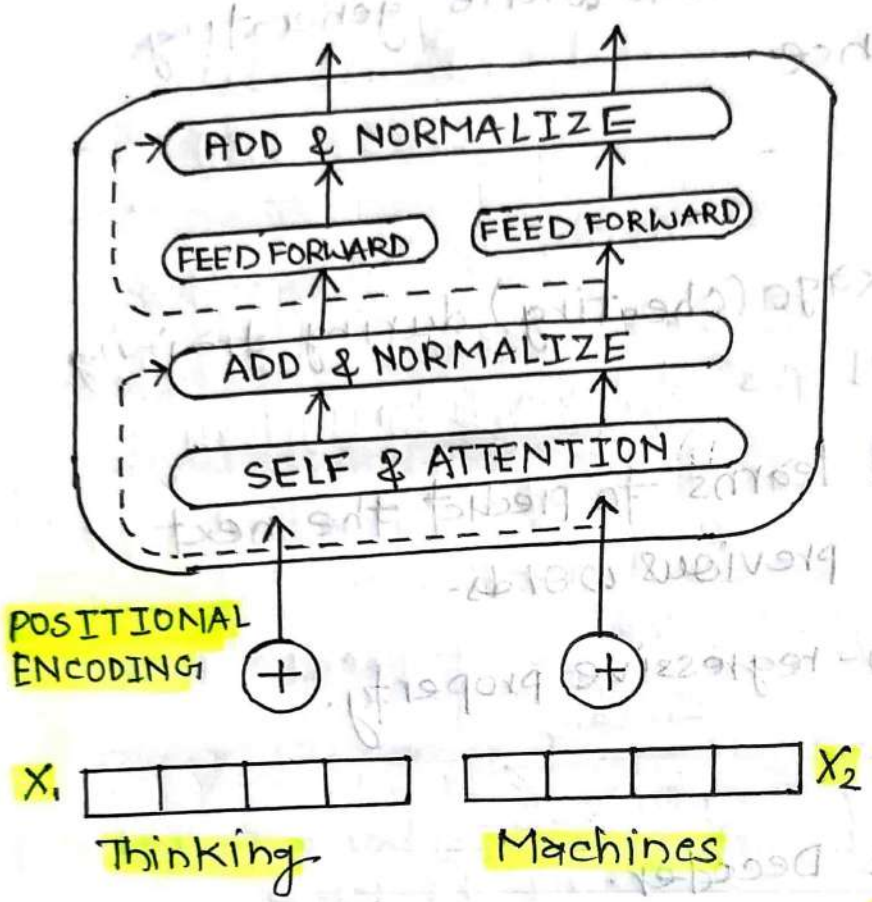
* stabilizes the values.
* Improves training speed & stability.

Output

- The final output is a context-Aware-Embeddings (Contextual Embedding).
- Each word now contains information from the most relevant words in the sentence.
- Then output is passed to Feed Forward Neural Network (FFN) for further processing.

→ Feed Forward Neural Network (FFN)

It is the second main layer of the Transformer encoder that processes each token independently to learn deeper features & strengthen its contextual representation.



↳ Example

512
↓
2048
↓
512

↳ Need of FFN

- Learns complex patterns & Features
- strengthens the contextual embedding produced by Self-Attention
- Process each token independently (no communication bet' tokens)

- Expansion (512 → 2048): Learns complex features
- Compression (2048 → 512): Returns to the original embedding size.

↳ Add & Norm (After FFN)

FFN output + original Input → Layer Normalization

- **Purpose**
  - * Preserves original information (Residual Connection)
  - * Stabilizes training (Layer Normalization)
- **Result**
  This completes one Encoder Block.

→ Masked Multi-Head self Attention

It is the attention mechanism used in the decoder that prevents a word from seeing future words while generating the output sequence.

↳ Need:

- Prevents data leakage (cheating) during training
  ↳ Model sees future words during training
- Ensures the model learns to predict the next word using only previous words.
- Maintains the auto-regressive property.

↳ Use

- Used only in the Decoder.
- Applies a mask ($-\infty$) before softmax so future words receive 0 attention

↳ Example

Sentence: "Please study Hard"

While predicting "study", the model can attend to:
Please
Study

- Hard is not shown to model because it a future word.

Raw attention score

1	0	1
1	1	2
2	1	3

→ scaling

0.71	0	0.71
0.71	0.71	1.42
1.42	0.71	2.12

Masked Layer

0	$-\infty$	$-\infty$
0	0	$-\infty$
0	0	0

→

0.71	$-\infty$	$-\infty$
0.71	0.71	$-\infty$
1.42	0.71	2.12

→ Encoder-Decoder Attention (Cross Attention)

Cross Attention allows the decoder to attend to the encoder's output while generating each target word.

↳ Need

- connects the input sentence with the output sentence.
- Helps the decoder focus on the most relevant input words during translation or generation.

↳ Use

- Used After Masked self-Attention in the decoder.
- Query (Q) comes from the Decoder
- Key (K) & Value (V) come from the Encoder output.

↳ Example

sentence: "Please study"

Generating Hindi: "कृपया पढ़ाई करें"

While generating "पढ़ाई", the decoder attends to the encoder word "study".

TRANSFORMER

NON-AUTO REGRESSIVE (TRAINING)

During training, the transformer processes the entire target sequence in parallel using Teacher Forcing

↳ **Need**

- Faster training
- efficient GPU parallel computation

↳ **Cons**

- can cause data leakage without masking
- Depend on Teacher Forcing, creating a mismatch with interface

↳ **Example**

Target sentence : कृपया पढ़ाई करो

All words (कृपया, पढ़ाई, करो) are fed to the decoder simultaneously during training.

AUTO REGRESSIVE (TESTING)

During testing, the Transformer generates one word at a time, feeding each predicted work back as input for the next prediction.

↳ **Need**

- Future words are unknown during inference.
- Produces natural sequential text generation.

↳ **cons**

- slower because generation is sequential.
- Errors can accumulate if an early prediction is incorrect.

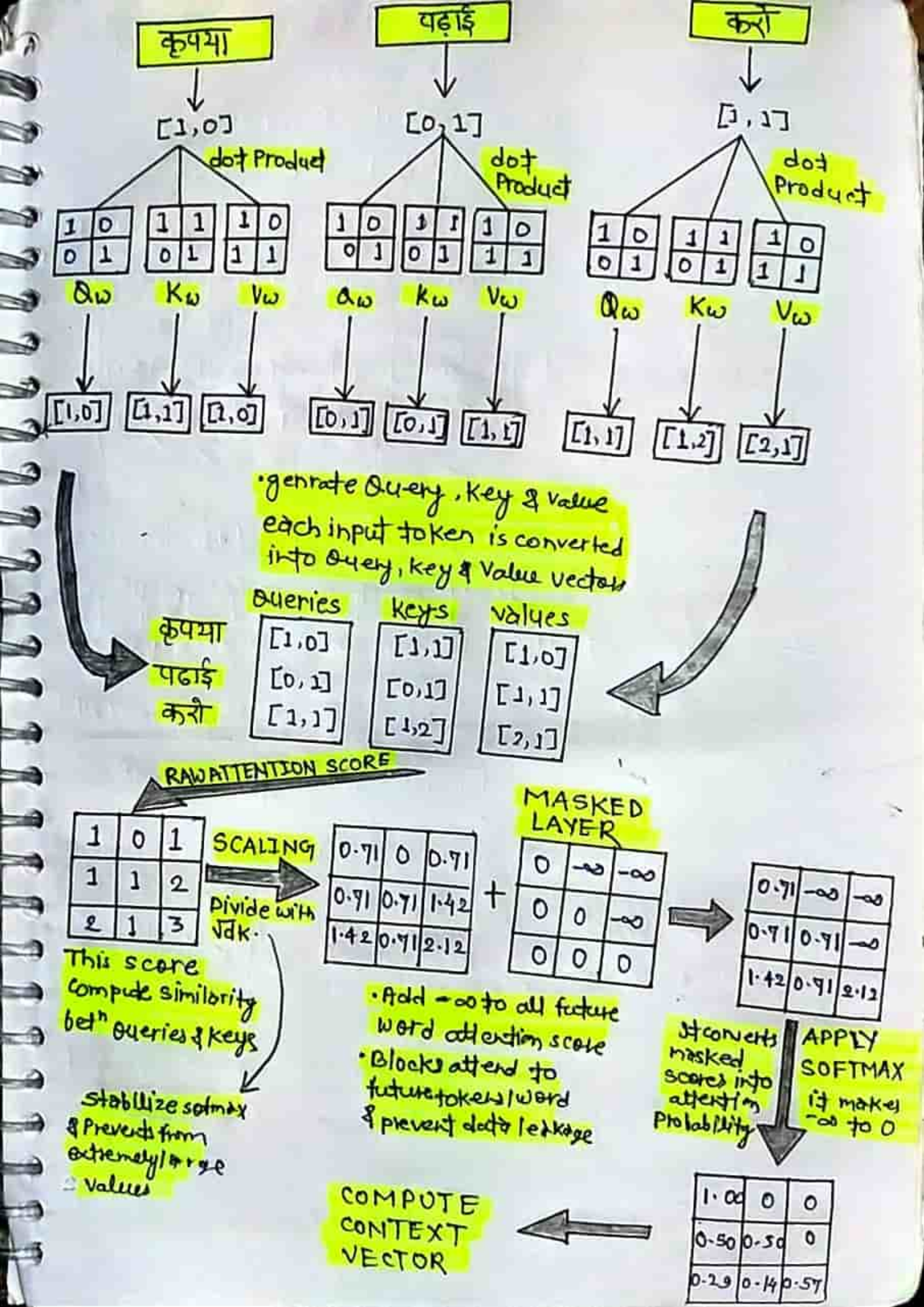
↳ **Example**

Input: "Please study"

Generation:

<START> → कृपया → पढ़ाई → करो → <END>

*Each word is generated using only the previously generated word


कृपया
पढ़ाई
करो
[1,0]
[0,1]
[1,1]
dot Product
dot Product
dot Product
Qw
Kw
Vw
Qw
Kw
Vw
Qw
Kw
Vw
[1,0]
[1,1]
[2,0]
[0,1]
[0,1]
[1,1]
[1,1]
[1,2]
[2,1]
•genrate Query, Key & Value
each input token is converted
into Query, key & Value vector
कृपया
पढाई
करो
Queries
[1,0]
[0,1]
[1,1]
keys
[1,1]
[0,1]
[1,2]
values
[1,0]
[1,1]
[2,1]
RAW ATTENTION SCORE
1 0 1
1 1 2
2 1 3
SCALING
Pivide with √dk.
0.71 0 0.71
0.71 0.71 1.42
1.42 0.71 2.12
+
MASKED LAYER
0 -∞ -∞
0 0 -∞
0 0 0
0.71 -∞ -∞
0.71 0.71 -∞
1.42 0.71 2.12
This score
Compute similority
bet" Queries & keys
Stabilize softmax
& Prevents from
extremely large
values
• Add -∞ to all future
word attention score
• Blocks attend to
future tokens/word
& prevent data leakage
It converts
masked
scores into
attention
Probability
APPLY SOFTMAX
it makes
-∞ to 0
1.00 0 0
0.50 0.50 0
0.29 0.14 0.57
COMPUTE
CONTEXT
VECTOR